A Priority-Based Global Routing for multi-Industrial Autonomous Vehicles

Zekuan Fang, Kaihao Lan, Mengpeng He, Tianhao Jiang, Zheng Yang, Liang Xie, Weihang Pan, Ziqi Wang, *Student Member, IEEE*, and Binbin Lin

***Abstract*—Autonomous driving is increasingly deployed for cargo transport in ports and mines, yet system-level efficiency still hinges on coordinated routing of multiple autonomous vehicles (AVs). Prior work largely focuses on Intersection Management (IM) and lacks global optimization aligned with industrial throughput. Inspired by warehouse multi-robot planning, we analyze multi-AV routing in relation to Multi-Agent Path Finding (MAPF), but note that standard MAPF relies on spatiotemporal discretization incompatible with mainstream AV stacks. We analyze the theoretical linkage between operational efficiency and global routing, and introduce IDS, a practical multi-AV routing method derived from this analysis. IDS conforms to AV system requirements on search procedures and output representations, and achieves global coordination via an efficiency-driven routing priority scheme. To our knowledge, IDS is the first multi-AV routing approach tailored to real AV systems that explicitly targets industrial operational efficiency. Simulation experiments using real-world port maps and AV parameters show that IDS improves crane efficiency by an average of 21.06% over Dijkstra, the routing method currently deployed in the corresponding ports. IDS also satisfies real-time constraints, computing concurrent paths for up to 110 AVs within one second. These results indicate that IDS enables scalable, efficiency-oriented coordination for industrial autonomous driving.**

***Note to Practitioners*—This paper is motivated by automated transportation in ports and investigates global routing for multiple autonomous vehicles in large, closed industrial environments. We conduct systematic observations and analyses of large-scale unmanned port operations and find that overall efficiency often lags behind human-operated workflows, primarily because current industrial-grade autonomous driving algorithms lack sufficient flexibility and timeliness to manage immediate and incipient congestion. Accordingly, we focus on the Routing Module of autonomous driving systems, aiming to enhance operational efficiency through coordinated optimization of global multi-AV paths. To our knowledge, prior multi-agent routing studies rarely target large industrial unmanned settings and seldom optimize explicitly for operational efficiency while validating on real autonomous driving systems. In contrast, our work is designed under deployability constraints and integrated into a production autonomy stack (with an architecture akin to Apollo). For validation, we build a simulation platform aligned with actual port processes, faithfully reproducing end-to-end container transfer executed by unmanned vehicles under Terminal Operating System dispatch. Experiments employ authentic dispatch instruction files, high-definition maps, and vehicle dynamics models parameterized by empirically measured vehicle properties. Results demonstrate significant gains in automated operational efficiency while meeting real-time requirements for industrial deployment, establishing the engineering feasibility and practical applicability of the proposed method. Future work will prioritize field deployment and conduct systematic parameter tuning informed by empirical evidence collected during testing.**

***Index Terms*—Autonomous Driving, Industrial Application, Multi-agent, Global Routing.**

Binbin Lin is the corresponding author. Email: binbinlin@zju.edu.cn.

This work was supported in part by Yongjiang Talent Introduction Programme (2022A-240-G).

Z. Fang, W. Pan, Z. Wang, and B. Lin are with the School of Software Technology, Zhejiang University, Ningbo 315100, China (e-mail: 12351005@zju.edu.cn; panweihang@zju.edu.cn; wangziqi0312@zju.edu.cn; binbinlin@zju.edu.cn).

L. Xie is with the School of Computer Science and Technology, Zhejiang University of Technology, Hangzhou 310014, China (e-mail: lilydedbb@gmail.com).

K. Lan, M. He, T. Jiang, and Z. Yang are with FABU Technology Co., Ltd., Hangzhou 310012, China (e-mail: lankaihao@fabu.ai; hemengpeng@fabu.ai; jiangtianhao@fabu.ai; yangzheng@fabu.ai).

I. Introduction

ALTHOUGH autonomous driving has been widely deployed in large-scale industrial scenarios (e.g., mines and ports) and has substantially reduced labor costs, considerable potential remains to improve operational efficiency through coordinated routing of multiple autonomous vehicles (AVs). To address this bottleneck, existing scheduling strategies primarily optimize vehicle passage sequences at intersections. This line of research develops high-efficiency intersection management (IM) strategies by fully leveraging AV cooperation capabilities [6]–[8], thereby increasing road-network throughput. However, IM methods overlook key operational factors in large industrial environments and lack global vehicle coordination. By focusing exclusively on the local environment [36] and disregarding scenario-specific characteristics—such as the locations and operating durations of working machines—these approaches improve localized traffic efficiency but do not necessarily translate those gains into overall operational efficiency [42]. Moreover, IM methods are constrained by the coverage of vehicle-to-vehicle (V2V) communication, rendering locally scoped scheduling strategies vulnerable to failure in large-scale scenarios [41].

Building on successful robot-management practices in warehouse logistics [1]–[4], recent studies formulate multi-AV routing in large-scale, closed industrial scenarios as a Multi-Agent Path Finding (MAPF) problem and apply classical MAPF methods to solve it [5]. However, when deployed within real-time autonomous driving systems, these methods exhibit significant limitations: MAPF algorithms generally fail to reflect the intrinsic characteristics and operational constraints of autonomous driving stacks [19], [20]. Classical MAPF approaches rely on a spatiotemporal discretization assumption [47], wherein the environment is gridded uniformly and agents are abstracted as point entities with binary motion (i.e., wait or move to the next vertex per time step). Even state-of-the-art MAPF studies, whether reinforcement learning-based

[39], [40] or heuristic-based [45], are typically framed under this assumption. Safe-Interval-Path-Planning (SIPP) [35], a variant of MAPF, introduces continuity only in the temporal dimension, yet still presumes a discretized spatial environment. By contrast, industrial autonomous driving systems require global routing that performs search on the topological graph constructed from High-Definition maps (HD maps) [38], and outputs global paths composed of lane segments, encompassing continuous driving and lane-change ranges. Moreover, industrial AVs characterized by large momentum cannot operate under binary-speed abstractions [46]. These discrepancies pose substantive challenges for directly adopting mainstream MAPF methods in industrial autonomous driving systems.

To address these limitations, we propose a novel multi-AV routing method, IDS, comprising three stages: **I**nformation Search, **D**irection Search, and **S**patiotemporal Search. The approach coordinates global paths for multiple AVs in industrial autonomous driving scenarios without discretizing space or time, while satisfying the interface and output requirements of downstream modules. In contrast to IM methods that optimize intersection-specific scheduling, IDS balances traffic flow across intersections from a global perspective and targets maximization of overall operational efficiency rather than intersection-level efficiency.

Our main contributions are as follows. (1) We introduce an efficiency-driven priority formulation and a three-stage graph-search algorithm that enables rapid global path coordination for large fleets of AVs. (2) To the best of our knowledge, we are the first to design a multi-AV routing method for industrial operation improvement that is applicable to real autonomous driving systems, and we conduct simulations on two real-world port maps using AV models parameterized by real data. (3) Experimental results demonstrate that, compared to existing methods, IDS can better optimize operational efficiency by coordinating multi-AV routing. Additionally, IDS can generate global paths for 110 AVs within one second.

II. Related Work

A. Intersection Management

As a research domain centered on AV scheduling, IM methods are designed to coordinate and optimize the traversal of multiple AVs through intersections by developing algorithmic frameworks that ensure safety, efficiency, and scalability [9], [10]. These approaches typically formalize intersection crossing as a constrained optimization or decision-making problem, wherein vehicles must resolve right-of-way conflicts, satisfy safety margins, and minimize delay under dynamic traffic conditions. To enable cooperative behavior, AVs rely on Vehicle-to-Vehicle (V2V) communication to exchange state and intent information—such as position, velocity, estimated arrival times, and planned maneuvers—and to anticipate potential conflicts [30], [32]. By integrating computational outputs from IM methods (e.g., priority scores, reservation schedules, or policy parameters), AVs collaboratively determine the sequence of passage through intersections and adapt their trajectories in real time to evolving traffic states [26]. For example, Li et al. [23] introduced a three-layer algorithm, PSL, inspired by MAPF, to efficiently and safely coordinate AVs at signal-free intersections in real time. Additionally, Zhou et al. [24] proposed a phase-time-traffic hypernetwork model utilizing a branch-and-bound search algorithm to optimize intersection automation strategies in mixed-traffic environments. Zhao et al. [25] proposed a centralized cooperative approach based on safe deep reinforcement learning to manage connected autonomous vehicles (CAVs) at unsigned intersections. Pan et al. [31] presented a two-stage framework using MAPPO and hybrid filtered beam search to coordinate CAVs across multiple unsignalized intersections for global efficiency.

However, the efficiency improvements at intersections achieved by these methods do not equivalently translate to enhanced operational efficiency in industrial scenarios. Moreover, methods lacking a global perspective may lead to system-wide optimization failure due to excessive local optimization.

B. Multi-Agent Path Finding

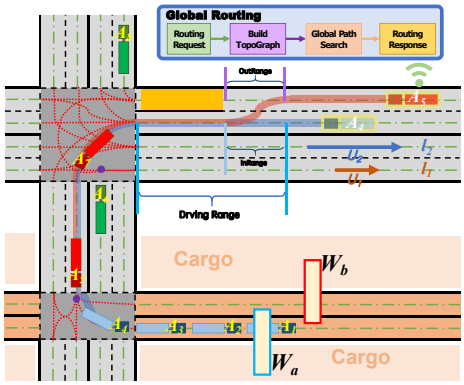
The MAPF problem concerns the computation of collision-free trajectories for agents operating on a shared graph or spatial domain, with the objective of navigating each agent from its start to its goal while optimizing collective performance criteria such as makespan, sum of costs, or throughput [11]–[13]. However, the implementation of classic MAPF methods, such as Conflict-Based Search (*CBS*) [47] and Priority-Based Search (*PBS*) [27], relies on the discretization assumption of space and time. Even recent MAPF methods [14], [15], such as *Lacam* [16], continue to operate within discretized environments, leveraging grid or graph abstractions and time-stepped conflict detection to bound computational complexity and ensure tractable search. Mainstream industrial autonomous driving frameworks, such as Autoware [21] and Apollo [22], require the Routing Module to provide global paths consisting of lane sections. The downstream modules need to control AVs' behavior in each lane based on lane-section information, such as continuous driving range, lane-change type, etc. [17], [18]. The differences between autonomous-driving global routing and MAPF in topological-graph representation and output format make it difficult to deploy MAPF methods in the Routing Module. In addition, SIPP [35], [46], a variant of MAPF, uses safe intervals to solve conflicts between agents in continuous time, but it is still limited to discrete spatial representations and requires agents to start/stop moving instantaneously. Therefore, designing a multi-AV routing method represents a significant and novel challenge.

III. Problem Formulation

A. Preliminaries

This paper addresses multi-AV routing for cargo transportation in large industrial scenarios, where AVs are Level 4 (L4) autonomous delivery vehicles [43], [44], such as intelligent container trucks (ICTs). Upon receiving transport instructions from the industrial operating system, AVs travel to designated locations to pick up or deliver cargo. The objective of multi-AV routing is to maximize scenario-level operational efficiency by coordinating global paths. Using the widely adopted autonomous driving framework **Apollo** [22] as an illustrative

reference, we define the general concepts pertinent to industrial operations and autonomous driving as follows:



Name	Symbol	Shape
Autonomous Vehicle	A	
Real Lane	l_r	
Virtual Lane	l_v	
Intersection	int	
Block	-	
Lane Connection	-	
Stop Point	-	
Global Path	π	
Working Machine	W	
Working Lane	l_w	
Cargo Area	-	

Fig. 1. Illustration of autonomous driving in large industrial scenarios.

- **TopoGraph:** In autonomous driving, a TopoGraph is a topological graph constructed from HD-map data and used by AVs to compute global paths to their destinations. Nodes represent lanes (with varying lengths), and edges represent connectivity between lanes. Further details of the TopoGraph are provided in Subsection III-B.
- **Real Lane and Virtual Lane:** Lanes in the HD map are categorized into **real lanes** and **virtual lanes**. As illustrated in Figure 1, green dashed lines denote real lanes, denoted by l_r, along which AVs travel in a straight path, while red dashed lines denote virtual lanes, denoted by l_v, which represent admissible traversal trajectories through intersections.
- **Working Machine:** In industrial scenarios, the working machine, W, denotes the destination of AVs, such as excavators in mines and cranes in ports. AVs receive task instructions and autonomously navigate to the location of the corresponding working machine.
- **Working Lane:** The working lane, l_w, is a real lane where AVs interact with working machines—for example, lanes in mine loading zones and port berth areas. As shown in Figure 1, green dashed lines with an orange background typically sustain long queues of AVs awaiting operations and are restricted to AVs assigned to the corresponding machine; AVs destined for other machines are not permitted to traverse them.
- **Block:** A block denotes an area where AVs are not allowed to traverse or where static obstacles are present. As illustrated in Figure 1, the yellow square marks an impassable region with no connectivity to adjacent lanes.
- **V2V Communication:** When AVs interact, the V2V Communication Module determines their behavior using a predefined priority hierarchy and prevents deadlock [28], [29]. As illustrated in Figure 1, the higher-priority vehicle (A_7) is granted non-disruptive right-of-way at intersections, while the lower-priority vehicle (A_6) proactively yields by decelerating.
- **Global Routing:** In autonomous driving, global routing refers to the process by which the Routing Module computes the information required for the AV to navigate to its destination, including the global path π composed of the sequence of lane sections $\{\mathcal{L}_1, \mathcal{L}_2, \dots\}$ and additional information required by downstream modules, such as the vehicle priority P_v. Further details of global routing are provided in Subsection III-B.
- **Operational Efficiency:** Operational efficiency is a key metric for evaluating performance in industrial scenarios, such as excavator loading and unloading throughput in mines and crane container-handling efficiency in ports.

To improve readability, Table I summarizes the core notation used in the main text, while the supplementary material provides a more complete notation list. For range notation, the superscripts m and r distinguish HD-map ranges from route-output ranges. The role subscript D denotes a drivable range for HD-map ranges and a driving range for route-output ranges, while O and I denote outgoing and incoming ranges, respectively. For indices, j is reserved for lane identifiers, α and β for pairwise agents or AVs, k for rank within an AV group, and w for working-machine groups.

TABLE I
CORE NOTATION USED IN THE MAIN TEXT.

Used in	Symbol	Description
AV system	A, $\mathcal{A}$	AV and AV group.
AV system	W	Working machine serving as an AV destination.
AV system	l_r, l_v, l_w	Real lane, virtual lane, and working lane.
Index	j, k, w	Lane identifiers, AV rank, and working-machine group index.
Routing rep.	$\mathcal{G}$, $\mathcal{G}_s$, $\mathcal{G}_p$	Global TopoGraph, AV-specific sub-TopoGraph, and prune-TopoGraph.
Routing rep.	V, E	TopoNode and TopoEdge.
Routing rep.	S_j	Length of lane j.
Routing rep.	$R^m_{D,j}$, $R^m_{O,j\to j_1}$, $R^m_{I,j\to j_1}$	HD-map drivable, outgoing, and incoming ranges.
Routing rep.	π, $\mathcal{L}$	Global path and its lane section.
Routing rep.	R^r_D, R^r_O, R^r_I	Route-output driving, outgoing, and incoming ranges.
Proof	t_a, t_e	Predicted arrival time and earliest interaction start time.
Proof	T_o, T_s	Operation duration and slack time.
Proof	B, T_f, t_{loss}	Behavior decision, total time cost, and yielding time loss.
Method	P_r, P_{int}, P_v	Routing priority, intersection priority, and vehicle priority.
Method	χ_H, χ_s, ϕ_p, ϕ_d	HPA indicator, final priority indicator, queue penalty, and direction-flow adjustment factor.
Method	H, Q	High-priority approximation parameter and maximum lane-change parameter.
Method	C_c, C_e, C_n	Current-node, edge, and next-node costs in graph search.
Method	u, $\mathcal{U}$	Recommended speed and optional speed set.
Method	γ, J_c, J_{int}	Spatiotemporal trajectory, chasing-collision judge, and intersection-collision judge.
Complexity	$\mathcal{V}_s$, N_{req}	Search-node space and number of routing requests.
Complexity	V_p, V_{seg}, o_p	Number of vertices in $\mathcal{G}_p$, number of nodes on a road segment between adjacent intersections, and maximum out-degree of $\mathcal{G}_p$.

B. Global Routing in Autonomous Driving

This subsection uses the widely adopted autonomous driving framework **Apollo** [22] as a representative case to elucidate the global routing workflow implemented in its Routing Module. It is important to note that global routing is not limited to computing a single end-to-end path from the origin to the destination. Rather, it also encompasses the construction of the underlying topological graph, the preprocessing and validation of request data, and the generation of actionable outputs for downstream modules. As shown in Figure 1, the Routing Module receives routing requests generated by the upstream Messaging Module. Each request includes the vehicle identifier, block metadata, start coordinates (x_s, y_s), end coordinates (x_e, y_e), the vehicle heading angle θ, and other relevant information. In this context, (x, y) denotes Universal Transverse Mercator (UTM) coordinates [48], and θ is defined as the angular deviation between the AV's heading direction and the UTM easting axis.

In response to routing requests, the Routing Module first constructs a global TopoGraph $\mathcal{G}$ together with multiple sub-TopoGraphs $\{\mathcal{G}_{s_1}, \mathcal{G}_{s_2}, \dots\}$. The global TopoGraph encodes the topology of the drivable area for AVs, where nodes represent lanes (both real and virtual) and edges indicate connectivity between adjacent lanes. To build $\mathcal{G}$, the module processes the complete set of lane data $\mathbf{L}$ from the HD-map. Each lane j is described by $\mathbf{L}_j$, which includes its identifier j, lane length S_j, HD-map drivable range $R^m_{D,j} = [0, S_j]$, and a set of connectivity relations $\mathcal{C}_j = \{\text{Con}_{j,j_1}, \dots\}$. Each connection Con_{j,j_1} specifies the connected lane $\mathbf{L}_{j_1}$, the connection type CT (e.g., straight-through, left lane-change, right lane-change), the permitted HD-map outgoing range $R^m_{O,j\to j_1}$ on lane j, and the permitted HD-map incoming range $R^m_{I,j\to j_1}$ on lane j_1. In this notation, R consistently denotes a range; the superscript m indicates that the range is stored in the HD-map, and the role subscripts D, O, and I denote drivable, outgoing, and incoming ranges, respectively. A TopoNode is defined as

$$V = \left[j, R^m_{D,j}, \mathcal{C}_j, \dots\right], \qquad V \in \mathcal{V}, \tag{1}$$

and a TopoEdge is represented as

$$E = \left[j, j_1, R^m_{O,j\to j_1}, R^m_{I,j\to j_1}, CT, \dots\right], \qquad E \in \mathcal{E}. \tag{2}$$

The global TopoGraph for autonomous driving is constructed by connecting all nodes with directed edges:

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}). \tag{3}$$

In addition to the global TopoGraph, the Routing Module constructs an AV-specific sub-TopoGraph $\mathcal{G}_s$ when processing a routing request. Each routing request contains block information, which specifies non-drivable lane segments in the form of lane identifiers and blocked arc-length ranges. To construct $\mathcal{G}_s$, the Routing Module maps each blocked range onto the corresponding TopoNode in $\mathcal{G}$. As illustrated in Figure 2, if a blocked range covers only part of a TopoNode, the original node is split at the block boundaries so that the blocked and unblocked ranges are represented by separate TopoNodes. The blocked TopoNodes or blocked sub-ranges are then removed, and the remaining TopoNodes and their valid inherited TopoEdges form the AV-specific sub-TopoGraph. Thus, $\mathcal{G}_s$ represents the drivable topological graph available to a specific AV under the block information contained in its current request. A pruning-related TopoGraph construction used by the proposed IDS method will be introduced later in Subsection VII-A.

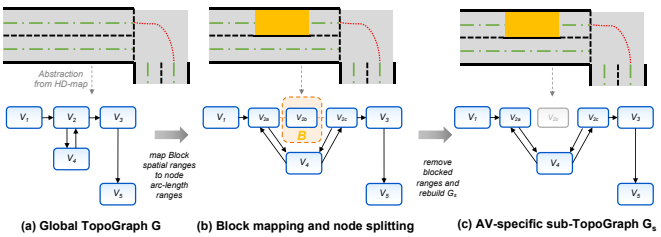


Fig. 2. Construction of an AV-specific sub-TopoGraph from the global TopoGraph according to request-level block information.

Subsequently, the localization function maps the start and end positions from UTM coordinates (x, y) to the topological format (j, s), where j denotes the lane identifier and $s \in [0, S_j]$ is the arc-length coordinate along that lane. Graph search algorithms (e.g., *Dijkstra*, *A**) are then employed to compute the minimum-cost global path from the start lane to the end lane. The global path π is defined as

$$\pi = \{\mathcal{L}_1, \mathcal{L}_2, \dots\}, \tag{4}$$

where each lane section $\mathcal{L}$ is represented as

$$\mathcal{L} = [j, R^r_D, R^r_O, R^r_I, u, \dots]. \tag{5}$$

Here, $\mathcal{L}$ denotes a lane section in the routing output rather than a lane in the HD map. The superscript r indicates route-specific ranges computed by the Routing Module. In this representation, j denotes the lane identifier, R^r_D specifies the range that the corresponding AV will traverse, R^r_O and R^r_I define the permissible ranges for the Planning Module to control lane exiting and entering, respectively, and u indicates the recommended speed, which the AV should approach without exceeding. Finally, the Routing Module transmits the global path π to the Planning Module by generating routing responses. These responses also include auxiliary information required by other modules, such as the vehicle priority P_v for the V2V Communication Module. **Note that global routing must be completed within a short time budget—typically under one second—to ensure timely consumption by downstream processes.**

C. Limitations of MAPF Methods in Autonomous Driving

Although our task formulation also involves computing global paths for multiple agents, the distinctive characteristics of autonomous driving—combined with the operational requirements of large-scale industrial deployments—render the path-finding problem fundamentally different from general MAPF settings.

In general MAPF settings, time is discretized into time steps, and the environment is modeled as a topological graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ denotes the set of locations and $\mathcal{E}$ denotes the set of feasible transitions between nodes. As

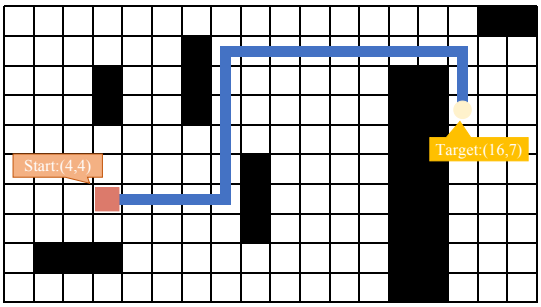


Fig. 3. General setting of Multi-Agent Path Finding.

illustrated in Figure 3, the environment is typically partitioned into uniformly sized grid cells, where each cell corresponds to a node (identified by its coordinates), and edges indicate permissible moves between adjacent cells. Given N agents $\mathcal{A} = \{A_1, A_2, \ldots, A_N\}$, each agent A_α is assigned a start and a goal location. At each time step, A_α may either remain at its current node or move to an adjacent node. The MAPF objective is to compute a set of global paths $\Pi = \{\pi_1, \pi_2, \ldots, \pi_N\}$ such that every agent reaches its goal without colliding with any other agent:

- **Global Path:** The global path of agent A_α is defined as

$$\pi_\alpha = \{V_\alpha^1, V_\alpha^2, \ldots, V_\alpha^\tau, \ldots, V_\alpha^{T_\alpha}\}, \tag{6}$$

  where V_α^τ denotes the location of A_α at time step τ, and T_α is the time step at which A_α reaches its goal.
- **Collision avoidance:** A solution is collision-free if, for all distinct agents $A_\alpha, A_\beta \in \mathcal{A}$ and for all time steps τ: (1) node conflict is avoided: $V_\alpha^\tau \neq V_\beta^\tau$; and (2) edge conflict is avoided: $(V_\alpha^\tau, V_\alpha^{\tau+1}) \neq (V_\beta^{\tau+1}, V_\beta^\tau)$.

The quality of a solution is evaluated by the makespan, defined as the latest arrival time among all agents:

$$T_{\max} = \max(T_1, T_2, \ldots, T_N). \tag{7}$$

The optimization objective is to minimize $T_{\max}$. Based on the above, global routing for autonomous driving fundamentally differs from classical MAPF in the following respects:

- **Topological Difference:** Autonomous driving typically constructs a TopoGraph from HD maps, whereas MAPF discretizes the environment into a grid. Consequently, the underlying topologies differ substantially. In autonomous driving, TopoNodes (Eq. 1) represent continuous, drivable lane segments, including both real and virtual lanes, together with lane-type attributes and associated semantics. In MAPF, by contrast, TopoNodes correspond to discrete spatial positions (grid cells or coordinates). A particularly salient distinction arises at intersections: AVs must adhere to predefined reference trajectories (i.e., virtual lanes) rather than transitioning to adjacent grid cells as in MAPF.
- **Output Difference:** As specified in Eq. 4 and Eq. 6, the routing output in autonomous driving is a sequence of lane sections, whereas MAPF produces an agent's grid location at each time step. For large vehicles with substantial momentum and variable speeds, the Planning Module cannot operate on binary, discrete-time actions (i.e., stopping or moving to an adjacent grid cell). Instead, it requires continuous, lane-centric trajectories.

In summary, mainstream MAPF methods—such as the classical *CBS* [47] and the advanced *Lacam* [16]—primarily target the efficient computation of collision-free routes. However, their conflict formulations, which rely on environmental discretization and small-motion assumptions for agents, are misaligned with the requirements of industrial autonomous driving. In addition, the computational cost of computing collision-free routes scales strongly with the size and density of the environment; methods that perform well in structured warehouse settings often struggle to satisfy the stringent real-time constraints of large-scale industrial deployments (e.g., ports and mining sites). **Consequently, a central challenge is to develop multi-AV routing approaches that are both computationally efficient and tailored to autonomous driving, thereby achieving efficient collaboration among multiple AVs in industrial scenarios.**

D. Priority-based Routing and Slack Time

Priority-based Planning [33] is widely recognized as an efficient paradigm for multi-agent planning, capable of rapidly generating globally feasible solutions at relatively low computational cost, and has therefore been extensively adopted in industrial applications. Consequently, we employ Priority-based Planning as the core strategy for our multi-AV routing method. The effectiveness of this strategy, however, critically depends on a well-designed priority scheme. In this subsection, we therefore investigate how routing priorities influence operational efficiency in industrial settings.

For an AV A en route to a working machine W, let T_o denote the operation duration that W spends on A, and t_a be the predicted arrival time at the destination when A follows the global path π, which we approximate by summing over lane segments:

$$t_a = \sum_{\mathcal{L}_\ell \in \pi} \frac{s_n - s_0}{\tilde{u}}, \quad R_D^r(\mathcal{L}_\ell) = [s_0, s_n], \tag{8}$$

where $\tilde{u} = u$ if a recommended speed is specified in $\mathcal{L}_\ell$; otherwise, $\tilde{u}$ equals the maximum speed of the lane associated with $\mathcal{L}_\ell$. We define $\mathbf{T}$ to be the nondecreasing sequence of earliest interaction start times for all AVs destined for W:

$$\mathbf{T} = [t_e^1, t_e^2, \ldots, t_e^k, \ldots], \tag{9}$$

where t_e^k denotes the earliest time at which A_k can interact with W, given recursively by $t_e^k = \max(t_a^k, t_e^{k-1} + T_o)$. If the sequence $\mathbf{T}$ satisfies

$$\forall t_e^k \in \mathbf{T}, \quad t_a^{k+1} - t_e^k \leq T_o, \tag{10}$$

then each successive AV arrives no later than the completion of the preceding operation. This no-idle service condition keeps W continuously occupied and maintains the optimal operational efficiency attainable under the duration T_o.

Mainstream autonomous driving frameworks typically employ static graph-search algorithms within their Routing Modules—for example, *Dijkstra* in Autoware [21] and *A** in

Apollo [22]—to compute global paths. These algorithms use a greedy strategy that selects the shortest global path to the destination, i.e., minimizes the arrival time t_a. Although this approach can improve the efficiency of an individual AV, global paths associated with different destinations may interact and interfere with one another, ultimately degrading system-level operational efficiency. For instance, as illustrated in Figure 1, A_4 will be delayed by the queue at W_a and will subsequently block A_5's access to W_b for an extended period. This interaction substantially reduces the operational efficiency of W_b. Therefore, local routing decisions should account for downstream interactions to improve system-level performance, rather than solely minimizing individual travel time. From a system-efficiency perspective, A_4 should yield at an earlier stage so that A_5 can follow its globally shortest path, enabling W_b to commence its operation as early as possible. In this case, the brief yielding maneuver does not delay A_4's arrival beyond the completion of the preceding operation at W_a; therefore, W_a remains continuously occupied and retains its optimal operational efficiency.

In summary, we introduce a novel routing indicator, termed **slack time**, to prioritize the routing of AVs. For the AV A_k with rank k, the slack time T_s^k is formally defined as

$$T_s^k = t_e^{k-1} + T_o - t_a^k. \tag{11}$$

To establish the theoretical basis for slack-time-based priority design, we compare the two feasible yielding decisions for a pairwise routing conflict over one priority-update interval, defined as the interval between consecutive priority computations. Within this interval, the interaction schedules of unaffected AVs and the relative arrival ordering are treated as unchanged, and operation durations are represented by a common value T_o.

Theorem 1 (Slack-Time-Based Pairwise Priority Ordering)**.** *Consider a **routing conflict** between two AVs A_α and A_β, whose destination working machines are W_α and W_β, respectively. Let $T_f^{\alpha>\beta}$ denote the overall operational time cost when A_β yields to A_α, and let $T_f^{\beta>\alpha}$ denote the cost under the reverse decision. If $T_s^\alpha > T_s^\beta$, the two yielding decisions satisfy*

$$\begin{aligned} W_\alpha = W_\beta &\implies T_f^{\alpha>\beta} \le T_f^{\beta>\alpha}, \\ W_\alpha \neq W_\beta &\implies T_f^{\alpha>\beta} \ge T_f^{\beta>\alpha}. \end{aligned}$$

Consequently, the cost-minimizing pairwise ordering assigns A_α no lower routing priority than A_β in the same-destination case and no higher routing priority than A_β in the different-destination case.

Theorem 1 is formalized and proved as follows:

Consider two AVs, A_α and A_β, each navigating toward its respective working machine, W_α and W_β, with predicted arrival times t_a^α and t_a^β, respectively. Suppose a conflict is anticipated along their future trajectories. Let B_α and B_β denote the behavior decisions issued by the V2V Communication Module for A_α and A_β at the conflict time. Specifically, $B_\alpha > B_\beta$ indicates that A_β decelerates to a stop to yield to A_α, whereas $B_\beta > B_\alpha$ indicates that A_α decelerates to a stop to yield to A_β. Let t_{loss} denote the yielding time loss, assumed to satisfy

$$t_{loss} \ll T_o, \quad t_{loss} \ll t_a. \tag{12}$$

Using the common operation duration T_o, we discuss the case with $T_s^\alpha > T_s^\beta$.

- If $W_\alpha = W_\beta$, then A_α and A_β have the same destination. We obtain:

$$t_e^{\alpha-1} = t_e^{\beta-1} \xrightarrow{T_s^\alpha > T_s^\beta} t_a^\alpha < t_a^\beta. \tag{13}$$

Given the vehicle behaviors determined by the V2V Communication Module, we compare the overall operational time cost T_f under the two yielding decisions. For $B_\alpha > B_\beta$, the time cost $T_f^{\alpha>\beta}$ is given by:

$$T_f^{\alpha>\beta} = \max\Big(\max(t_a^\alpha, t_e^{\alpha-1}+T_o)+T_o, t_a^\beta + t_{loss}\Big)+T_o. \tag{14}$$

Furthermore, the same destination and mutual interaction between A_α and A_β imply temporal proximity in their arrival times, i.e., $|t_a^\alpha - t_a^\beta| \ll T_o$. Therefore, we obtain:

$$\max(t_a^\alpha, t_e^{\alpha-1} + T_o) \ge t_a^\alpha \tag{15}$$

$$T_f^{\alpha>\beta} \xrightarrow{|t_a^\alpha - t_a^\beta| \ll T_o} \max(t_a^\alpha, t_e^{\alpha-1} + T_o) + 2T_o. \tag{16}$$

Likewise, for $B_\beta > B_\alpha$, $T_f^{\beta>\alpha}$ can be calculated as:

$$T_f^{\beta>\alpha} = \max(t_a^\beta, t_e^{\beta-1} + T_o) + 2T_o. \tag{17}$$

Since $t_e^{\alpha-1} = t_e^{\beta-1}$ and $t_a^\alpha < t_a^\beta$, we directly derive:

$$T_f^{\alpha>\beta} \le T_f^{\beta>\alpha}. \tag{18}$$

Eq. 18 establishes the same-destination relation in Theorem 1: for minimizing the overall operational time cost, assigning A_α a routing priority no lower than that of A_β, corresponding to decision $B_\alpha > B_\beta$ between the two alternatives, is optimal.

- If $W_\alpha \neq W_\beta$, we obtain:

$$\begin{aligned} T_f^{\alpha>\beta} =& \max(t_a^\alpha, t_e^{\alpha-1} + T_o) + T_o + \\ & \max(t_a^\beta + t_{loss}, t_e^{\beta-1} + T_o) + T_o \\ T_f^{\beta>\alpha} =& \max(t_a^\beta, t_e^{\beta-1} + T_o) + T_o + \\ & \max(t_a^\alpha + t_{loss}, t_e^{\alpha-1} + T_o) + T_o \end{aligned} \tag{19}$$

then,

$$\begin{aligned} T_f^{\alpha>\beta} - T_f^{\beta>\alpha} =& \max(t_a^\alpha, t_e^{\alpha-1} + T_o) + \\ & \max(t_a^\beta + t_{loss}, t_e^{\beta-1} + T_o) - \\ & \max(t_a^\beta, t_e^{\beta-1} + T_o) - \\ & \max(t_a^\alpha + t_{loss}, t_e^{\alpha-1} + T_o) \end{aligned} \tag{20}$$

Next, we discuss the sign conditions of T_s^α and T_s^β:
(1) If $0 > T_s^\alpha > T_s^\beta$, then,

$$\begin{aligned} T_f^{\alpha>\beta} - T_f^{\beta>\alpha} =& (t_a^\alpha + t_a^\beta + t_{loss}) - \\ & (t_a^\beta + t_a^\alpha + t_{loss}), \end{aligned} \tag{21}$$

Thus, $T_f^{\alpha>\beta} - T_f^{\beta>\alpha} = 0$, and we obtain:

$$T_f^{\alpha>\beta} = T_f^{\beta>\alpha}. \tag{22}$$

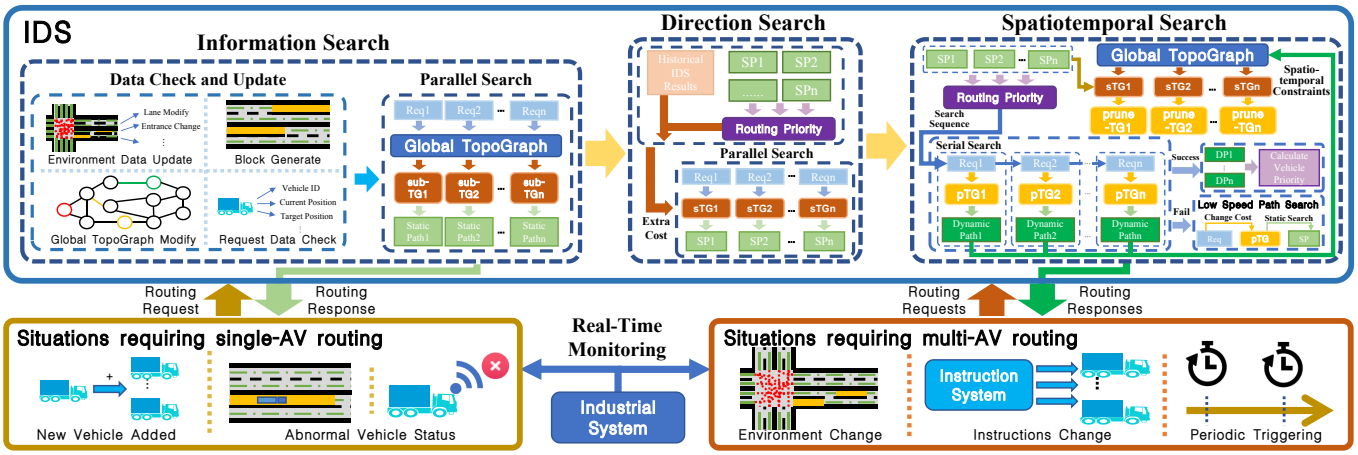


Fig. 4. Overall Framework of the IDS-Augmented Routing Module. The upper panel presents the global routing process driven by IDS, and the lower panel characterizes the triggering conditions of IDS for both single-AV and multi-AV routing schemes.

(2) If $T_s^\alpha \geq 0 > T_s^\beta$, then,

$$T_f^{\alpha>\beta} - T_f^{\beta>\alpha} = (t_e^{\alpha-1} + T_o + t_a^\beta + t_{loss}) - (t_a^\beta + t_e^{\alpha-1} + T_o). \tag{23}$$

Thus, $T_f^{\alpha>\beta} - T_f^{\beta>\alpha} > 0$, and we obtain:

$$T_f^{\alpha>\beta} > T_f^{\beta>\alpha}. \tag{24}$$

(3) If $T_s^\alpha > T_s^\beta \geq 0$, then,

$$T_f^{\alpha>\beta} - T_f^{\beta>\alpha} = (t_e^{\alpha-1} + T_o + t_e^{\beta-1} + T_o) - (t_e^{\beta-1} + T_o + t_e^{\alpha-1} + T_o). \tag{25}$$

Thus, $T_f^{\alpha>\beta} - T_f^{\beta>\alpha} = 0$, and we obtain:

$$T_f^{\alpha>\beta} = T_f^{\beta>\alpha}. \tag{26}$$

Based on the above, if $W_\alpha \neq W_\beta$ and $T_s^\alpha > T_s^\beta$, decision $B_\alpha > B_\beta$ has no lower overall operational time cost than decision $B_\beta > B_\alpha$. Therefore, A_α has no higher routing priority than A_β, which establishes the different-destination relation in Theorem 1.

In summary, Theorem 1 establishes a conditionally optimal priority ordering for overall operational efficiency under the stated operating conditions, rather than an unconditional guarantee across arbitrary operating regimes. IDS uses this conditionally optimal relationship as the theoretical foundation for a practical priority rule. Because the operation duration T_o can vary substantially in real-world deployments, the priority formulation introduced below uses arrival-order information and an average operation duration instead of directly applying the unstable scheduling term T_s. Successive priority computations refresh the ordering as vehicle states and predicted arrival times evolve; abrupt changes within a single priority-update interval may temporarily reduce the representativeness of the current priority estimate. On top of this priority design, we coordinate multi-AV global paths using a three-stage graph search algorithm introduced below.

IV. Overall Framework

This section presents a request–response framework for global routing within the Routing Module, augmented by the IDS approach. The core of IDS is a practical priority formulation built on the conditionally optimal ordering established by Theorem 1, which coordinates the global routes of all AVs to improve overall operational efficiency while reducing computation time to meet real-time constraints. As illustrated in Figure 4, the upper panel delineates a three-stage search for request processing: **Information Search** preprocesses input data and computes static global paths; **Direction Search** refines routes based on the priority metric to balance prospective intersection flows; and **Spatiotemporal Search** locally adjusts global paths via a serial time-dependent search, assigning speeds and vehicle priorities in the responses.

The lower panel enumerates all situations that trigger routing requests in a real-time autonomous driving system. Because an AV's driving state is highly sensitive to environmental perturbations, the Routing Module may receive requests from upstream modules at arbitrary times. Moreover, as time advances, discrepancies inevitably accumulate between an AV's actual and predicted positions, necessitating periodic recomputation of the global route. Accordingly, we categorize routing triggers into two classes:

- **Multi-AV requests:** The Routing Module ingests information from all AVs and computes global routes using IDS. These requests are typically clock-triggered: all AVs periodically update their global routes at a frequency of $1/T_\omega$. In addition, environmental changes and instruction updates can also generate multi-AV requests.
- **Single-AV requests:** The Routing Module ingests information for a single AV and computes its global route via Information Search. Such requests are commonly triggered when an AV initiates a new task—for example, immediately after completing its current task and being assigned a new destination. Abnormal AV states (e.g., abnormal application of the global route in the Planning Module) can likewise trigger single-AV requests.

V. Information Search

The Information Search stage validates request data, updates environmental information, constructs the global TopoGraph and per-AV sub-TopoGraphs, and computes each AV's static path π_s. As illustrated in Figure 4, the Routing Module receives HD-map data and routing requests from all AVs; invalid requests return an error response, while valid requests are processed in parallel by constructing sub-TopoGraphs and performing static path searches. Both the Information Search and Direction Search stages employ the *Dijkstra* algorithm for static path finding. In Information Search, the inter-node cost is the travel distance from the current node to the next node:

$$C_n = C_c + C_e + \omega_s(s_n - s_0), \tag{27}$$

where C_c and C_n denote the costs from the source node to the current node V_c and to the next node V_n, respectively; C_e is the edge cost between them and is typically higher for left or right lane changes. $R_D^r(\mathcal{L}) = [s_0, s_n]$ denotes the route-specific traversed range on the lane associated with V_c, and ω_s is the distance-to-cost conversion coefficient. It follows that the Information Search stage yields static paths of minimum length for all AVs to reach their destinations.

VI. Direction Search

Direction Search improves high-traffic intersection efficiency by adjusting the global routes of AVs with low routing priority. As shown in Figure 5, Information Search selects the shortest trajectory as each AV's global path (the purple trajectory), but an individually optimal path is not necessarily system-wide optimal. If intersections int_1, int_2, and int_3 experience heavy demand near the busy cargo area, the purple route traverses all three and further exacerbates congestion. **In contrast to existing IM methods that primarily optimize the crossing order of AVs at high-traffic intersections, Direction Search proactively reduces future intersection loads by routing detours.** The blue trajectory in Figure 5 illustrates a detoured global path. Although this path is substantially longer, the AV encounters only one high-traffic intersection (int_3). Under the conditions of Theorem 1, an AV with sufficient estimated slack can take a longer path without violating the no-idle service condition in Eq. 10, thereby preserving the optimal operational efficiency of its associated working machine. The resulting reduction in intersection interference for AVs with high routing priority can improve machine efficiency by decreasing t_a and t_{loss}. Consequently, Direction Search computes priorities and adapts AV costs to balance intersection traffic flows dynamically.

A. High-Priority Approximation Based on Rank

Routing priority determines the order in which AVs search their global paths under the Priority-based Planning strategy. The theoretical analysis in Subsection III-D establishes the relationship between an AV's routing priority and its slack time; however, slack time can be unstable during operation. In closed industrial transportation environments, the relative arrival order of AVs bound for the same destination tends to remain stable over a short priority-update interval. Specifically,

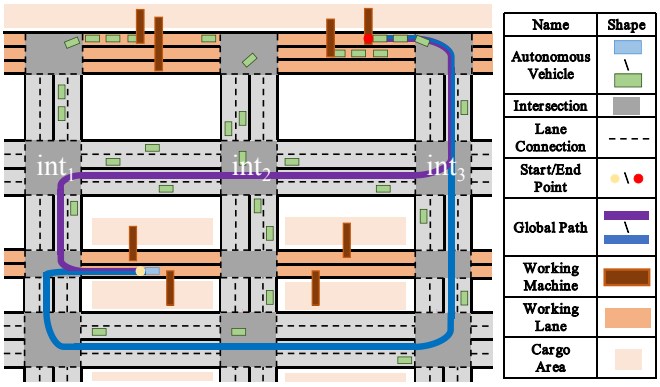


Fig. 5. Illustration of Direction Search detouring to mitigate intersection congestion.

for all AVs in the transportation region, we group them by their target working machines and sort each group in ascending order of arrival time t_a. Let $\mathcal{A}_w$ denote the set of AVs destined for working machine W_w,

$$\mathcal{A}_w = \{A_1, A_2, \ldots, A_k, \ldots\}, \tag{28}$$

where k indicates that AV A_k has rank k in $\mathcal{A}_w$ by arrival time. We use this group rank as a robust surrogate for slack time and update the ranks whenever priorities are recomputed. We then reformulate the calculation of T_s^k for A_k as:

$$\begin{aligned}
T_s^k &= t_e^{k-1} + T_o^{k-1} - t_a^k \\
&= \max(t_a^{k-1}, t_e^{k-2} + T_o^{k-2}) + T_o^{k-1} - t_a^k \\
&= \max(t_a^{k-1}, T_s^{k-1} + t_a^{k-1}) + T_o^{k-1} - t_a^k \\
&= T_s^{k-1} + t_a^{k-1} + T_o^{k-1} - t_a^k - \min(0, T_s^{k-1}), \\
T_s^k - T_s^{k-1} &= t_a^{k-1} + T_o^{k-1} - t_a^k - \min(0, T_s^{k-1}), \\
T_s^k - T_s^1 &= t_a^1 - t_a^k + \sum_{h=1}^{k-1} T_o^h - \sum_{h=1}^{k-1} \min(0, T_s^h), \\
T_s^k &= -t_a^k + \sum_{h=1}^{k-1} T_o^h - \sum_{h=1}^{k-1} \min(0, T_s^h).
\end{aligned} \tag{29}$$

Furthermore, we derive a high-priority approximation (HPA) for T_s. Since

$$-\sum_{h=1}^{k-1} \min(0, T_s^h) \geq 0, \tag{30}$$

the slack time T_s^k of AV A_k can be reformulated as:

$$T_s^k \xrightarrow{\text{HPA}} -t_a^k + \sum_{h=1}^{k-1} T_o^h. \tag{31}$$

Based on the theoretical analysis in Subsection III-D, the routing-priority paradigm is formulated as:

$$P_r^k = -\omega_{t_1} \max(-t_a^k + \sum_{h=1}^{k-1} T_o^h, 0) - \epsilon k, \tag{32}$$

where ω_{t_1} denotes the time-to-priority conversion coefficient, and ϵ is a small positive constant. To enhance the stability

of the routing-priority paradigm, we further apply the HPA to Eq. 32. For AV $A_k \in \mathcal{A}_w$ with rank k, we introduce the HPA parameter H to obtain a stabilized priority representation:

$$P_r^k \xrightarrow{\text{HPA}} -\omega_{t_1}\chi_H(k)\max\Big(-t_a^k+(k-1)T_{o'},0\Big)-\epsilon k, \quad (33)$$

where $T_{o'}$ denotes the average operating duration of the working machine, and $\chi_H(k)$ denotes the HPA indicator,

$$\chi_H(k)=\begin{cases}0, & k\le H,\\ 1, & k>H.\end{cases} \quad (34)$$

The use of $T_{o'}$, rank ordering, and the HPA indicator translates the slack-time relationship into an implementable priority rule. This approximation closely reflects the slack-time ordering when the leading ranks remain stable over a priority-update interval and actual operation durations remain close to $T_{o'}$. Additionally, to ensure the validity of Eq. 12 by reducing the likelihood of intersection congestion caused by vehicle queuing (e.g., A_4 blocking A_5 from entering the intersection in Figure 1), we incorporate an additional penalty term into the routing priority. Specifically, for the working lane l_w associated with the working machine W_w, its maximum capacity N_w is given by:

$$N_w=\frac{s_w-s_0}{L_A+L_{safe}}, \quad (35)$$

where s_w denotes the position of W_w on l_w, s_0 denotes the initial position at which the AV enters l_w, L_A is the AV length, and L_{safe} is the minimum safe distance. Consequently, the routing priority P_r with the queue penalty ϕ_p is:

$$P_r^k=-\omega_{t_1}\Bigg[\chi_s(k)\max\Big(-t_a^k+(k-1)T_{o'},0\Big)+\phi_p(k)\Bigg]-\epsilon k, \quad (36)$$

where

$$\chi_s(k)=\begin{cases}0, & k\le \min(H,N_w),\\ 1, & k>\min(H,N_w).\end{cases} \quad (37)$$

$$\phi_p(k)=\begin{cases}0, & k\le N_w,\\ \max(t_a^{k-N_w}+T_{o'}-t_a^k,0), & k>N_w.\end{cases} \quad (38)$$

B. Cost Design

Guided by the priority paradigm defined in Eq. 36, we redesign the cost function used in Direction Search. The objective is to achieve cross-intersection flow balancing at the network level: AVs with low routing priority are steered toward suboptimal detours, thereby reducing the interaction-induced delays experienced by AVs with high routing priority at intersections. Specifically, for AV A_k, the virtual lane cost in the sub-TopoGraph is redesigned as:

$$C_n=C_{c_v}+C_e+\omega_s\phi_d(P_r)(s_n-s_0), \quad (39)$$

where C_{c_v} denotes the accumulated cost from the source node to the current node along the virtual lane, ϕ_d denotes the direction-flow adjustment factor, which is adjusted according to the routing priority,

$$\phi_d(P_r)=-\omega_d P_r\sum_{d_h\neq d_0}X(d_h), \quad d_h,d_0\in\mathbf{D}, \quad (40)$$

where ω_d is the priority-to-cost conversion coefficient, $X(d_h)$ denotes the number of AVs expected to enter the intersection from direction d_h, and d_0 is the direction of the predecessor lane of the current virtual lane. Note that $X(d_h)$ is derived from the most recent trigger of IDS. Based on Eq. 39, Direction Search recomputes static global paths for all requests. AVs with low routing priority are routed to static paths that minimize traversal of high-traffic intersections, thereby reducing interference with the global paths of AVs with high routing priority, shortening the latter's arrival times, and improving overall operational efficiency.

VII. Spatiotemporal Search

Spatiotemporal Search transforms static paths into dynamic ones by applying a time-based graph search algorithm under spatiotemporal constraints. Building upon the globally directed static path, we introduce localized adjustments to endow the global route with behaviors such as overtaking and lane-change yielding, and we assign a speed attribute to each lane section in global paths. Consequently, it is necessary to introduce the temporal dimension and abandon the greedy strategy at this stage of path search, so as to explore feasible local spatiotemporal trajectories induced by variations in speed and lane-change locations around the priority-aware static path. During path search, the set of optional speeds for an AV on a real lane is constrained by operational conditions and environmental characteristics. As shown in Figure 1, the set of optional speeds for a lane is denoted by $\mathcal{U}$. In the experiments, these sets are configured on a lane-by-lane basis according to the speed attributes in each port's HD map and the corresponding AV operating constraints, and the resulting lane-level settings remain fixed throughout the comparative evaluation. The mean speed $\overline{u}$ for the lanes (l_1,l_2) within the same road shows a decreasing trend from the innermost to the outermost lane; specifically, $\overline{u}_2>\overline{u}_1$. In typical road configurations and HD-map data, lanes such as l_2 connect to a larger number of virtual lanes, enabling more turning options at intersections. As a result, these lanes are often critical for AVs to reach their destinations rapidly. In contrast, outermost lanes such as l_1 have a lower likelihood of interaction with AVs in adjacent lanes, making them more suitable for deceleration and collision avoidance. Based on the above, the cost formulation for the time-dependent graph search at this stage is defined as:

$$C_n=C_c+C_e+\omega_{t_2}\frac{s_n-s_0}{u_\nu}, \quad u_\nu\in\mathcal{U}_c, \quad (41)$$

where ω_{t_2} is the coefficient that converts time to cost, and $\mathcal{U}_c$ represents the set of optional speeds of the lane corresponding to the current node V_c. The time-based cost design implies that the search space preserves nodes that reach the same location at different times, enabling exploration of spatiotemporal trajectories that satisfy the constraints within the search space.

However, augmenting search nodes with a temporal dimension expands the search space exponentially, which severely degrades search efficiency. To mitigate this, Spatiotemporal Search adopts a serial time-dependent search to coordinate global routes according to routing priority, using TopoGraph pruning, lane-change constraints, and periodic search-space clearing to reduce computational overhead.

A. Prune-TopoGraph

Different from the AV-specific sub-TopoGraph $\mathcal{G}_s$, which is constructed by the Routing Module from request-level block information, the prune-TopoGraph $\mathcal{G}_p$ is an IDS-specific search-space reduction built from $\mathcal{G}_s$ according to the static path produced by Direction Search. To reduce exploration in non-critical areas while refining lane-change granularity, this stage applies pruning to the sub-TopoGraph prior to graph search. As shown in Figure 6, the prune-TopoGraph $\mathcal{G}_p$ provides a bounded corridor for dynamic search around the priority-aware static path and refines the lane-change intervals at a finer granularity. Specifically,

- For the real lane section $\mathcal{L}_r$ in the static path π_s, the prune-TopoGraph retains this lane and all real lanes that lie on the same road as this lane.
- For the virtual lane section $\mathcal{L}_v$ in the static path π_s, the prune-TopoGraph retains this lane and all virtual lanes within the same intersection that share identical in-roads and out-roads with this lane.
- For edges whose type is not straight-in, the edge is partitioned into multiple sub-edges to enable lane changes from any position. The original HD-map lane-change ranges R_O^m and R_I^m are converted into continuous sub-ranges (each no smaller than the minimum admissible lane-change range).

The above pruning criteria intentionally restrict the search space, but they do not determine the priority order or the macroscopic detour decision. Spatiotemporal Search is performed serially according to routing priority, and $\mathcal{G}_p$ is constructed only after Direction Search has produced a priority-aware static path. For AVs with high routing priority, the Direction Search cost remains close to the original shortest-path cost, so the constructed prune-TopoGraph preserves the local solution space around the path that is most relevant to the system-level objective. For AVs with low routing priority, IDS does not require individual shortest-path optimality. As indicated by the slack-time analysis in Subsection III-D, assigning such AVs to suboptimal detours or local yielding behaviors does not necessarily reduce the operating efficiency of their working machines when sufficient slack exists; instead, it can reduce interference with AVs assigned high routing priority. Therefore, prune-TopoGraph should be understood as a corridor-level reduction for local spatiotemporal coordination rather than a complete search over all detour routes in the AV-specific sub-TopoGraph. It preserves local alternatives needed for conflict resolution, including same-road lane changes, equivalent virtual-lane connections for the same intersection movement, and refined lane-change ranges. This restriction is necessary for real-time routing: prune-TopoGraph trades full detour completeness for computational tractability while remaining aligned with the priority-based scheduling objective of IDS.

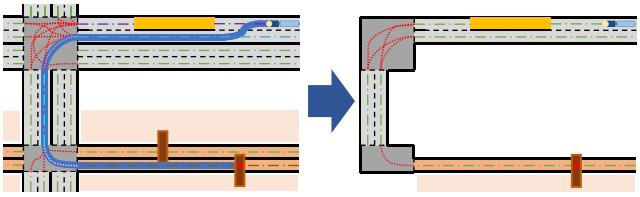


Fig. 6. Change in drivable areas induced by TopoGraph pruning. The left panel shows the drivable area of the sub-TopoGraph, while the right panel shows the drivable area of the prune-TopoGraph.

B. Spatiotemporal Constraint

Priority-based Planning requires the graph-search procedure to account for the spatiotemporal trajectories of AVs with higher routing priority. In our task setting, two types of collisions are considered at this stage.

- **Chasing collision**: When the graph search process of AV A_α explores node V_c corresponding to the real lane l_r (a non-working lane), let the entry position be s_0^α, the earliest entry time be t_0^α, and the exit position be s_n^α. If A_α selects speed u_α as the traveling speed on l_r, the exit time t_n^α is given by:

$$t_n^\alpha = t_0^\alpha + \frac{s_n^\alpha - s_0^\alpha}{u_\alpha}. \tag{42}$$

The spatiotemporal trajectory γ_r^α of A_α on l_r is:

$$\gamma_r^\alpha = [(s_0^\alpha, t_0^\alpha), (s_n^\alpha, t_n^\alpha)]. \tag{43}$$

For AV A_β with routing priority P_r^β higher than that of A_α, suppose A_β follows a spatiotemporal trajectory γ_r^β on l_r. To determine whether γ_r^α and γ_r^β intersect, we use the cross-product test. Define

$$\begin{aligned}
\zeta_1 &= (s_0^\beta - s_0^\alpha)(t_n^\alpha - t_0^\alpha) - (t_0^\beta - t_0^\alpha)(s_n^\alpha - s_0^\alpha),\\
\zeta_2 &= (s_n^\beta - s_0^\alpha)(t_n^\alpha - t_0^\alpha) - (t_n^\beta - t_0^\alpha)(s_n^\alpha - s_0^\alpha),\\
\zeta_3 &= (s_0^\alpha - s_0^\beta)(t_n^\beta - t_0^\beta) - (t_0^\alpha - t_0^\beta)(s_n^\beta - s_0^\beta),\\
\zeta_4 &= (s_n^\alpha - s_0^\beta)(t_n^\beta - t_0^\beta) - (t_n^\alpha - t_0^\beta)(s_n^\beta - s_0^\beta),
\end{aligned} \tag{44}$$

if

$$(\zeta_1\zeta_2 \le 0) \wedge (\zeta_3\zeta_4 \le 0), \tag{45}$$

then γ_r^α and γ_r^β intersect, indicating a chasing collision between A_α and A_β when A_α travels at speed u_α. Consequently, A_α cannot depart from position s_0^α at time t_0^α with speed u_α, and the graph search must not expand this successor node.

- **Intersection collision**: When the graph search of A_α visits node V_{v_1} corresponding to the virtual lane l_{v_1}, let t_{int}^α denote A_α's arrival time at the intersection. The intersection priority P_{int} is then defined as:

$$P_{\text{int}}^\alpha = P_r^\alpha - \omega_{t_1} t_{\text{int}}^\alpha, \tag{46}$$

where P_r denotes the routing priority. Consider an AV A_β with higher routing priority that passes through the same

intersection from virtual lane l_{v_2}, and suppose the lateral distance between l_{v_1} and l_{v_2} is less than the minimum AV safety interval. Then, if the logical XNOR condition

$$(P_{\text{int}}^{\alpha} < P_{\text{int}}^{\beta}) \odot (t_{\text{int}}^{\alpha} \leq t_{\text{int}}^{\beta}), \tag{47}$$

holds, an intersection collision is detected. Consequently, A_α cannot traverse the intersection at time t_{int}^{α}, and the graph-search process must not expand this successor node.

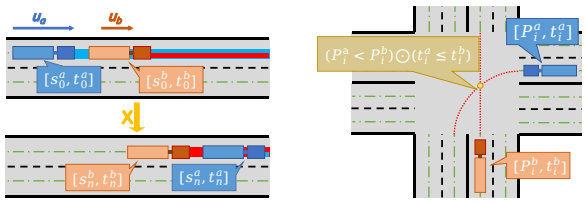


Fig. 7. Illustration of chasing and intersection collisions in global routing.

C. Process

Based on the prune-TopoGraph and spatiotemporal constraints, the Spatiotemporal Search stage proceeds as follows. First, for each AV, a prune-TopoGraph $\mathcal{G}_p$ is constructed concurrently across all AVs, using the AV's sub-TopoGraph and static path. Next, all routing requests entering this stage have their routing priority P_r recomputed and are then sorted in descending order. Subsequently, guided by the Priority-based Planning strategy, a novel time-dependent graph-search algorithm sequentially computes each AV's collision-free global path by exploring the spatiotemporal domain with an explicit time dimension, while reducing computational complexity to polynomial time by limiting lane-change frequency and periodically resetting the search space.

Algorithm 1 details the graph-search procedure. M is a threshold specifying the maximum number of search iterations, preventing the procedure from entering an infinite loop or exceeding the time limit. During each search iteration, the algorithm selects the unvisited node with the minimum cost in search-node space $\mathcal{V}_s$, denoted by V_c, together with its associated attributes. If $\mathcal{V}_s$ is empty, the search has failed to find a valid path to the target node V_t. Conversely, if V_c matches the target node V_t, the search succeeds, and the global path π is reconstructed from the predecessor links of V_c. To curb exponential growth of the search space induced by the time dimension, the algorithm clears the search space whenever V_c is a virtual lane node. In Lines 11–15, when V_c is identified as a virtual lane node, the algorithm clears the current search-node space and reinserts V_c. If no termination condition is met, the algorithm then expands V_c to generate new search nodes. Line 21 enumerates all outgoing edges, and Line 22 identifies the successor node V_n associated with the selected edge e_η. Specifically, Lines 24–30 enforce that, when the cumulative count of left or right lane changes at V_c exceeds Q, the expansion step accepts only the outgoing edge classified as straight-in to the next node. Here, the node attribute $V_c.q$ records the accumulated number of non-straight transitions along the current partial path segment since the latest reset. Line 31 enumerates all admissible speeds associated with V_c, and Line 32 calculates the arrival time to the successor node using the chosen speed u_ν. Lines 34–39 evaluate, according to the constraints specified in Figure 7, whether proceeding to the successor node at speed u_ν would result in a spatiotemporal conflict with AVs assigned higher routing priority. For any u_ν satisfying the constraints, the algorithm evaluates the cost according to Eq. 41 and instantiates a new search node to be inserted into $\mathcal{V}_s$ for use in the subsequent iteration.

For requests that successfully pass Algorithm 1, a global path π is reconstructed from the resulting node sequence, and the vehicle priority P_v is determined by the intersection priority P_{int} of the intersection nearest to the AV's position:

$$\mathcal{L}_v^{\star} = \arg \min_{\mathcal{L}_v \in \pi^+(j_A, s_A)} d_\pi\big((j_A, s_A), \mathcal{L}_v\big), \quad P_v = P_{\text{int}}(\mathcal{L}_v^{\star}), \tag{48}$$

where (j_A, s_A) is the current topological position of the AV, with j_A denoting the current lane identifier and s_A denoting the arc-length coordinate along that lane. $\pi^+(j_A, s_A)$ denotes the untraveled portion of π, $d_\pi((j_A, s_A), \mathcal{L}_v)$ is the along-path distance from (j_A, s_A) to a virtual lane section $\mathcal{L}_v$, and $P_{\text{int}}(\mathcal{L}_v^{\star})$ represents the intersection priority associated with the nearest upcoming intersection. Because the nearest intersection is position-dependent, changes in the vehicle's position may modify the nearest intersection and, consequently, the vehicle priority. For requests for which Algorithm 1 does not find an admissible path within the restricted search space and iteration budget, IDS reassigns the corresponding AVs to low-speed lanes to reduce interference with higher-priority AVs. For these AVs, we revise the cost function in their prune-TopoGraphs:

$$C_n = C_c + C_e + \omega_s \overline{u}(s_n - s_0). \tag{49}$$

We then recompute the global path via *Dijkstra*. The vehicle priority value P_v for these AVs is assigned the maximum constant value. Finally, the IDS generates the routing response for each AV, comprising the global path π and the vehicle priority P_v.

D. Time Complexity Analysis

In the Spatiotemporal Search stage, augmenting node exploration with an explicit temporal dimension and relaxing the constraints of a traditional greedy strategy jointly induce an exponential growth in the number of search nodes. Concretely, when we enumerate feasible speed selections for each node and propagate their implied arrival times, a single candidate node can spawn multiple distinct arrival-time instances. On the search-node space $\mathcal{V}_s$ induced by the prune-TopoGraph $\mathcal{G}_p$, the total number of search nodes can be characterized as:

$$|\mathcal{V}_s| = 1 + o_p U + (o_p U)^2 + \cdots + (o_p U)^{V_p - 1}, \tag{50}$$

where U denotes the number of optional speed selections per TopoNode, V_p is the number of vertices in the prune-TopoGraph, and o_p is the maximum out-degree over all vertices in the prune-TopoGraph. The resulting $|\mathcal{V}_s|$ grows

Algorithm 1: Spatiotemporal Search Process

Input: (i) M, maximum number of search iterations; (ii) $\mathcal{V}_s$, search-node space; (iii) V_s, start node; (iv) V_t, target node; (v) $\mathcal{V}_{\text{int}}$, set of intersection vertices; (vi) Q, maximum number of lane changes; (vii) π, global path; (viii) $J_c(\gamma)$, chasing-collision judge; (ix) $J_{\text{int}}(t, V)$, intersection-collision judge;

Output: π: collision-free spatiotemporal path from V_s to V_t, or $\emptyset$ if no admissible path is found within the prescribed search limits

```
// Initialization
1  V_s ← ∅ ;
2  V_s ← V_s ∪ {V_s} ;
// Graph Search
3  for κ ← 1 to M do
4      if V_s ≠ ∅ then
5          V_c ← V_s.PopMin() ;
6          t_a^c ← V_c.ArrivalTime ;
7          C_c ← V_c.Cost ;
8          if V_c = V_t then
9              π ← ReconstructPath(V_c) ;
10             return π ;
11         else if V_c ∈ V_int then
12             Clear(V_s) ;
13             V_c.q ← 0 ;
14             V_s ← V_s ∪ {V_c} ;
15         end
16     else
17         return ∅ ;
18     end
19     E_c ← V_c.OutEdges ;
20     U_c ← V_c.Speeds ;
21     for e_η ∈ E_c do
22         V_n ← e_η.NextNode ;
23         C_e ← e_η.Cost ;
24         if e_η.CT ≠ straight-in then
25             if V_c.q > Q then
26                 continue ;
27             else
28                 V_c.q ← V_c.q + 1 ;
29             end
30         end
31         for u_ν ∈ U_c do
32             t_ν ← t_a^c + (s_n − s_0)/u_ν ;
33             γ_c ← [(s_0, t_a^c), (s_n, t_ν)] ;
34             if J_c(γ_c) then
35                 continue ;
36             end
37             if (V_n ∈ V_int) ∧ J_int(t_ν, V_n) then
38                 continue ;
39             end
40             V_n.Cost ← C_c + C_e + ω_{t_2}(s_n − s_0)/u_ν;
41             V_s ← V_s ∪ {V_n} ;
42         end
43     end
44 end
45 return ∅ ;
```

sharply with V_p; consequently, enumerating and traversing node arrival times under this regime becomes computationally prohibitive. **This growth is the main reason why the time-dependent search cannot be performed directly on the full sub-TopoGraph.** Compared with $\mathcal{G}_p$, the full sub-TopoGraph contains many more vertices and, more importantly, much larger local branching factors around intersections, where multiple incoming lanes, outgoing lanes, and virtual lane movements create a dense set of successor choices. The impact of this branching factor is captured by o_p. In the prune-TopoGraph, o_p is bounded by the retained corridor: real-lane expansions are limited to lanes on the same road, and virtual-lane expansions are limited to equivalent movements with the same in-road and out-road. In the full sub-TopoGraph, however, the corresponding maximum out-degree would be much larger because intersection nodes can connect to many turning movements and downstream roads. Even when the later lane-change cap and periodic clearing reduce the dependence on V_p to a linear factor, the larger out-degree still appears in both the search-node bound and the successor-expansion term, thereby multiplying the overall time complexity. Consequently, a routing response that can be generated within hundreds of milliseconds on the prune-TopoGraph may require tens of seconds or even minutes on the full sub-TopoGraph, which is incompatible with the real-time response requirement of the Routing Module. To curb this expansion, we enforce a cap on the maximum number of lane changes Q (Lines 24–30 in Algorithm 1) and periodically clear the search space (Lines 11–15 in Algorithm 1), which jointly bound the growth of $\mathcal{V}_s$:

$$|\mathcal{V}_s| = o_p^{V_{\text{seg}}-1} U^{Q-1} \frac{V_p}{V_{\text{seg}}}, \tag{51}$$

where V_{seg} denotes the number of nodes on a road segment between adjacent intersections. Consequently, the time complexity of the graph search can be expressed as:

$$\begin{aligned} \mathcal{T}(\mathcal{G}_p, N_{\text{req}}) &= O(N_{\text{req}})\, O(|\mathcal{V}_s|)\, O(N_{\text{req}} o_p U) \\ &= O\left(\frac{N_{\text{req}}^2 V_p o_p^{V_{\text{seg}}} U^Q}{V_{\text{seg}}} \right), \end{aligned} \tag{52}$$

where N_{req} represents the number of routing requests and $O(N_{\text{req}} o_p U)$ accounts for the time complexity of expanding new search nodes.

VIII. Experiment

To evaluate the effectiveness of the proposed method and its feasibility for deployment in real industrial autonomous driving systems, we conduct simulation experiments on a Linux-based workstation equipped with an 11th Intel Core i7 CPU (3.60 GHz) and 32 GB RAM. The simulation environment is instantiated from two real-world ports (Port M and Port Y) that have already adopted automated container transportation. For evaluation, IDS and all baseline methods are deployed within the Routing Module of the autonomous driving systems operating in these ports.

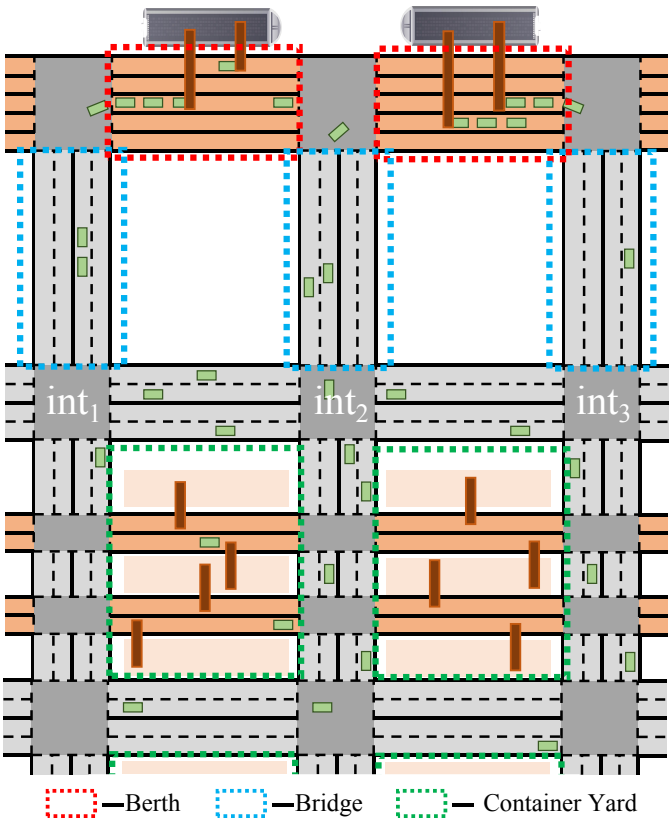


Fig. 8. Illustration of the port horizontal transport area.

A. Simulation Settings

The simulation platform used in the experiments is a self-developed traffic-flow simulation system specifically designed for automated driving scenarios in port terminals. The system supports explicit modeling of port operation processes and provides high flexibility in configuring terminal environments. It can be integrated with the Terminal Operating System (TOS) and is compatible with HD maps, shop-floor interaction systems, and multi-AV routing algorithms. AVs in the simulation are capable of performing fundamental automated driving behaviors, including trajectory planning, path following, and car-following, which are globally coordinated by a central dispatching system. This design ensures the realism and reliability of both AV kinematics and interactive behaviors, thereby providing a high-fidelity simulation environment for algorithm validation.

Figure 8 illustrates a representative structural layout of the autonomous driving area in the port, while the supplementary multimedia provides the structural layouts of the autonomous driving areas in Port M and Port Y. In the experiments, the drivable areas for AVs are restricted to specific regions, covering approximately 1.13 km^2 in Port M and 0.54 km^2 in Port Y. As exemplified in Figure 8, the area dedicated to automated container transportation mainly comprises three components:

- **Berths** serve as critical interfaces where vessels interact with port handling equipment. When an AV arrives at a berth, the crane performs container loading and unloading operations on the AV.
- **Container Yards** are operational areas in a port designated for storing containers. When an AV arrives at a container yard, the crane performs loading and unloading operations on the AV.
- **Bridges** serve as dedicated lanes that connect the landside (container yards) to the seaside (berths) of a port, and they are typically configured as long, straight alignments.

To enhance simulation realism, the AV models are calibrated against AVs operating in Port M and Port Y. Table II summarizes a subset of the model parameters, where Length and Width denote the vehicle length and width (in meters), and Safety Distance refers to the minimum allowable distance (in meters) between AVs enforced by the autonomous driving system. The observed energy-rate parameters in Table II are obtained from real-world vehicle operation records, each of which logs the operation duration, travel distance, and battery-percentage consumption of one AV. The mileage and time energy rates are reported as monthly average ratios, i.e., percentage consumption per kilometer and percentage consumption per hour, respectively. Based on these parameters, the simulated AVs exhibit high fidelity to their real-world counterparts in terms of shape, size, and driving trajectories. In our experimental setup, we simulate intelligent container trucks (ICTs) operating in Port M and intelligent guided vehicles (IGVs) deployed in Port Y. During the simulation, the autonomous driving system ingests preconfigured instruction files, thereby emulating the TOS issuing task commands to AVs. The Message Module translates TOS instructions into standardized requests and forwards them to the Routing Module, which computes routing responses to guide AVs to their assigned destinations. In addition, the V2V Module regulates interactive behaviors according to vehicle priorities, resolving potential conflicts and coordinating maneuvers among AVs.

TABLE II
KEY PARAMETERS OF AVS.

Type	Length	Width	Safety Distance	Observed Energy Rate	
				Mileage	Time
IGV	15.0	2.9	11.7	1.40	4.57
ICT	17.3	3.0	11.7	0.73	3.69

We design five port operational scenarios that reflect real-world settings and cover different scales in terms of the number of vessels and AVs. As shown in Table III, we configure three operational scenarios in Port M and two in Port Y, each with different numbers of cranes (working machines) and AVs. In these scenarios, AVs are tasked with transferring a prescribed number of containers for each vessel to designated destinations. To comprehensively evaluate IDS performance, we employ both randomly generated instructions and real historical instructions—collected in November 2024 for Port M and January 2025 for Port Y—in each scenario. Random instruction generation comprises two sampling steps. First, destination probabilities are determined by the proportions of tasks assigned to individual cranes, and the destination of each instruction is sampled accordingly. Second, after removing anomalous observations from long-term operation-

duration records collected at the two ports, we construct a port-specific empirical duration distribution and sample the duration of each crane operation from the corresponding distribution. All destination assignments and operation durations are generated and fixed before the simulation begins. A scenario-specific random seed is reused across all evaluated methods and repeated executions, ensuring that they receive the same complete instruction sequence. The instruction sequence and operation durations of each historical-instruction scenario are likewise fixed before simulation.

TABLE III
STATISTICS OF SIMULATION SCENARIOS.

Scenario	Vessels	Cranes	Containers	AVs
M1	1	13	5000	50
M2	2	18	10000/10000	60
M3	3	35	3000/5000/3000	110
Y1	1	10	5000	40
Y2	2	20	5000/3000	64

B. Baseline

To evaluate IDS under comparable routing-module interfaces, we compare it against four baseline methods that cover the routing strategies most closely related to our setting: deployed static routing, priority-based MAPF, and intersection-management (IM) scheduling.

- **Dijkstra:** A classical static graph-search method that computes the shortest path to a designated target under static environmental assumptions. This approach has been implemented in autonomous driving systems at Port M and Port Y, making its comparative evaluation highly relevant to practical deployment.
- **PBS:** A classical priority-based MAPF method that maintains a partial priority order over agents via a high-level DAG and plans individually optimal paths at the low level consistent with that order [27]. To make the comparison compatible with the routing representation used in this paper, conflicts in PBS are formulated using the spatiotemporal constraints defined in Subsection VII-B.
- **FIFO:** A classical IM method that assigns right-of-way strictly by arrival time at conflict points, independent of lane or movement, using a global queue and basic trajectory conflict checks [34].
- **PSL:** A state-of-the-art IM method that combines Safe Interval Path Planning (SIPP) [35] and linear programming. This method runs significantly faster than advanced Mixed Integer Linear Programming (MILP) methods, with only a slight compromise in solution quality.

The IM strategies are integrated with *Dijkstra*'s algorithm in the Routing Module, and vehicle behavior at intersections is governed by the computed vehicle priority P_v. Other recent MAPF algorithms are not directly used as baselines because their standard formulations do not share the same routing output and motion assumptions as the evaluated autonomous-driving Routing Module. Taking *Lacam* [16] as a representative example, these methods rely on spatiotemporal discretization to efficiently compute conflict-free paths; however, in our setting, autonomous driving alters the definition of conflicts and agent motion is non-binary, rendering these assumptions incompatible with a direct same-interface comparison. Moreover, MILP-based IM methods are not employed because their computational cost grows rapidly with fleet size [36], [37]. For example, the method in [36], a widely cited IM approach, achieves competitive efficiency for small autonomous-vehicle (AV) fleets, but its performance degrades substantially as the fleet size increases; the reported computation time exceeds 30 seconds once the number of AVs surpasses 40.

C. Coefficient and Parameter Settings

Table IV summarizes the coefficient settings used in our study. Specifically:

- The same value of the distance conversion coefficient ω_s is used in all static graph search processes.
- The triggering time T_ω is set identically for all methods that require periodic triggering.
- We analyze nearly one month of container-handling operations for all cranes at Port M. After data cleaning, the average operation duration per crane $T_{o'}$ is estimated to be approximately 93 seconds.
- Both parameters H and Q are set to 4, and their effects are examined in Subsection VIII-H.

TABLE IV
COEFFICIENT AND PARAMETER SETTINGS.

Coefficient	ω_s	ω_{t_1}	ω_{t_2}	ω_d	$T_{o'}$	T_ω	H	Q
Value	1	0.01	0.01	0.1	93 s	10 s	4	4

D. Evaluation Metrics

The operational efficiency of ports is directly reflected in container loading and unloading performance, i.e., crane efficiency. Crane efficiency is typically measured as the number of containers handled per unit time and is commonly reported in twenty-foot equivalent units per hour (TEU/hour). Note that, in our simulation, each crane transfers one forty-foot container per move (2 TEU). In addition to crane efficiency, we also compare the total operation time (hours) and the total travel distance (kilometers) in each scenario. These two metrics not only provide a more comprehensive characterization of port operational efficiency but also enable the estimation of AV energy consumption under different algorithmic configurations. To make this energy-related evaluation more explicit, we fit an empirical energy-consumption model using more than 10,000 real-world single-vehicle operation records collected in April 2026 from the two AV types considered in our experiments under comparable load conditions. Each record contains the vehicle energy consumption, driving duration, and travel distance. The monthly average ratios reported in Table II summarize the same type of raw records, whereas the coefficients in Eq. 53 are obtained by fitting the additive energy model to these triplet records. This data-driven formulation follows the common practice of estimating electric-vehicle energy consumption from measured operational data using

TABLE V
PERFORMANCE COMPARISON OF CRANE EFFICIENCY (TEU/HOUR) ACROSS SCENARIOS.

Methods	Scenario-Instruction									
	M1-Rand	**M1-Real**	**M2-Rand**	**M2-Real**	**M3-Rand**	**M3-Real**	**Y1-Rand**	**Y1-Real**	**Y2-Rand**	**Y2-Real**
Dijkstra	40.44	36.38	41.54	40.80	40.08	36.80	41.48	38.36	35.96	35.06
PBS	40.46	36.32	41.40	40.82	39.98	37.02	41.66	38.46	36.08	35.20
FIFO	41.20	36.20	43.98	39.98	42.14	39.84	38.02	39.22	33.52	34.78
PSL	40.82	37.14	45.28	41.08	42.28	40.54	41.04	39.43	34.46	35.79
IDS	**52.28**	**44.62**	**51.98**	**46.28**	**49.90**	**43.80**	**51.66**	**41.72**	**42.40**	**43.94**
Improvement	**+29.28%**	**+22.65%**	**+25.15%**	**+13.43%**	**+24.50%**	**+19.02%**	**+24.54%**	**+8.76%**	**+17.91%**	**+25.33%**

TABLE VI
PERFORMANCE COMPARISON OF TOTAL OPERATION TIME (HOURS) ACROSS SCENARIOS.

Method	Scenario-Instruction									
	M1-Rand	**M1-Real**	**M2-Rand**	**M2-Real**	**M3-Rand**	**M3-Real**	**Y1-Rand**	**Y1-Real**	**Y2-Rand**	**Y2-Real**
Dijkstra	24.85	27.56	81.30	82.50	25.55	29.53	30.36	32.65	28.13	29.06
PBS	24.84	27.60	81.40	82.48	25.67	29.40	30.21	32.49	28.03	28.99
FIFO	24.20	27.71	76.06	83.78	24.39	27.95	32.80	31.96	30.49	29.48
PSL	24.52	27.47	73.27	80.19	24.03	27.23	30.46	31.87	30.00	28.49
IDS	**19.20**	**22.53**	**64.68**	**75.62**	**20.59**	**23.03**	**24.26**	**30.01**	**23.95**	**23.63**
Improvement	**+22.74%**	**+18.25%**	**+20.44%**	**+8.34%**	**+19.41%**	**+22.01%**	**+20.09%**	**+8.09%**	**+14.86%**	**+18.69%**

TABLE VII
PERFORMANCE COMPARISON OF TOTAL TRAVEL DISTANCE (KILOMETERS) ACROSS SCENARIOS.

Method	Scenario-Instruction									
	M1-Rand	**M1-Real**	**M2-Rand**	**M2-Real**	**M3-Rand**	**M3-Real**	**Y1-Rand**	**Y1-Real**	**Y2-Rand**	**Y2-Real**
Dijkstra	6673.71	6820.13	27364.08	27165.08	15214.34	15619.63	4699.77	4802.71	8557.03	8667.57
PBS	6696.01	6903.01	27663.07	27261.48	15415.94	15632.03	4712.07	4837.01	8603.13	8714.31
FIFO	6701.30	6883.72	27845.53	27825.29	15444.31	15622.84	5081.13	4876.86	8657.09	8739.73
PSL	6723.24	6798.72	27886.99	27840.03	15474.32	15646.70	4959.21	4870.91	8604.22	8684.35
IDS	6947.04	7417.23	28075.48	28895.42	15540.24	15667.40	5319.99	5521.83	8892.57	8874.86
Improvement	**-4.10%**	**-8.75%**	**-2.60%**	**-3.33%**	**-2.14%**	**-0.31%**	**-13.20%**	**-14.97%**	**-3.92%**	**-2.39%**

regression-based models for energy prediction and energy-efficient routing [49]. Because detailed speed profiles, acceleration histories, and auxiliary-load measurements are not available in the scenario-level simulation outputs, we adopt an additive linear form based on the two aggregate variables that are consistently available in both the real records and the simulation: travel distance and operating time. This form also preserves additivity, allowing single-vehicle energy estimates to be aggregated to the task level. For each AV type, the real-world records are used to fit a separate pair of mileage- and time-related coefficients. As a scenario-level approximation, the scenario operation time is converted into aggregate vehicle-hours using the number of AVs in the scenario, because individual vehicle activity profiles are not available in the simulation outputs. Let μ index the routing method and σ index the simulation scenario in this evaluation. The task-level energy consumption of method μ in scenario σ is therefore estimated as

$$\hat{E}_{\mu,\sigma} = \rho_d^{c_\sigma} D_{\mu,\sigma} + \rho_t^{c_\sigma} N_\sigma T_{\mu,\sigma}, \tag{53}$$

where $D_{\mu,\sigma}$ is the total travel distance, $T_{\mu,\sigma}$ is the total operation time, N_σ is the number of AVs, c_σ denotes the AV type used in scenario σ, and $\rho_d^{c_\sigma}$ and $\rho_t^{c_\sigma}$ are the corresponding type-specific energy coefficients fitted from the real-world records.

E. Performance Comparison

Table V compares the crane efficiency achieved by IDS and the baseline methods, while Tables VI and VII report the total operation time and total travel distance, respectively. The suffixes "-Rand" and "-Real" denote scenarios using randomly generated and real historical instructions, respectively. "Improvement" denotes the relative change of IDS over *Dijkstra*, which is the routing method deployed in real-world autonomous driving systems; negative values in Table VII indicate increased travel distance. For each scenario–instruction configuration, all methods are evaluated using identical instruction sequences and operation durations. Repeated executions of each method use the same simulation-clock configuration, routing-trigger interval T_ω, and other simulation parameters, and yield identical crane efficiency, total operation time, and total travel distance. These repetitions serve as repeatability checks and are not treated as independent stochastic trials. Tables V, VI, and VII report the corresponding method comparisons for the evaluated configurations. From these results, we observe that:

- IM methods do not yield significant gains in crane efficiency. Their primary limitation is the lack of global dispatching: during the simulations, we frequently observed pairs of AVs that needed to reorder their intersection crossings but could not do so because the remaining approach distance was insufficient for safe trajectory adjustment. In contrast, IDS enables AVs to adapt their

global paths well before reaching intersections.

- IDS achieves the highest crane efficiency among all methods. Compared with *Dijkstra*, IDS improves crane efficiency by 24.28% under random-instruction scenarios and by 17.84% under real-instruction scenarios.
- Regarding energy consumption, each AV type is modeled with its own fitted coefficients. The mean values of the fitted coefficients are $\rho_d = 0.458\%$/km and $\rho_t = 1.132\%$/vehicle-hour, with a five-fold cross-validation MAE of 0.43% and RMSE of 0.55% on the real-world operation records. We then apply Eq. 53 to the total travel distance in Table VII, the total operation time in Table VI, and the AV count in Table III. Although IDS increases total travel distance by 5.57% on average, its reduction in aggregate vehicle-hours offsets part of the additional mileage. Consequently, averaged over all scenarios, the estimated energy consumption of IDS is 2.06% lower than that of *Dijkstra*, with reductions of 3.14% under random-instruction scenarios and 0.98% under real-instruction scenarios.

These findings indicate that IDS enables coordinated multi-AV routing through its prioritization strategy and three-stage search, thereby substantially improving operational efficiency.

F. Ablation Study

Table VIII reports crane efficiency under different stage combinations within IDS. Here, **I** denotes Information Search only, and **IS** denotes Information Search combined with Spatiotemporal Search. Direction Search depends on historical dynamic paths, whereas Spatiotemporal Search depends on static paths; therefore, **D**, **S**, **ID**, and **DS** cannot be evaluated in isolation. From Table VIII, we observe that:

- Spatiotemporal Search substantially improves performance by assigning vehicle priorities and speeds to AVs, thereby enabling overtaking and avoiding conflicts.
- Direction Search enhances crane efficiency under higher traffic volumes by adjusting the routing directions of AVs with low routing priority, thereby accelerating the arrival of AVs with high routing priority at their target machines.

TABLE VIII
CRANE EFFICIENCY (TEU/HOUR) UNDER DIFFERENT STAGE COMBINATIONS IN IDS.

Method	Scenario-Instruction			
	M1-Real	M3-Rand	Y1-Real	Y2-Real
I	36.42	39.94	38.40	35.16
IS	44.08	44.82	41.66	40.26
IDS	**44.62**	**49.90**	**41.72**	**43.94**

G. Runtime Analysis

Figure 9 compares the runtime of IDS and the baselines in two ports with different numbers of AVs. Although IDS is not the fastest algorithm, it can still perform global cooperative routing for 110 AVs in under 1 second. Moreover, as the number of AVs increases, the runtime of IDS does not exhibit an exponential surge; instead, its growth remains moderate, consistent with the theoretical analysis that the time complexity of IDS scales polynomially with the number of AVs. This behavior indicates predictable performance under increasing traffic demand, making IDS suitable for large-scale deployments where responsiveness and stability are essential. Taken together, these findings demonstrate that IDS strikes a practical balance between solution quality and computational efficiency.

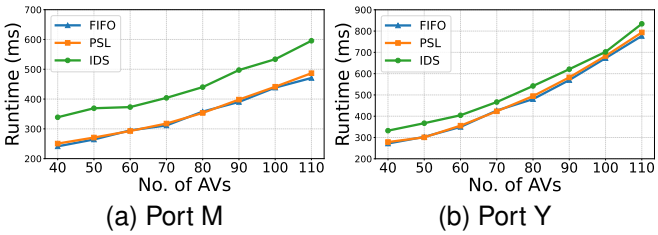


Fig. 9. Runtime comparison under different numbers of AVs.

Table IX further evaluates the computational effect of TopoGraph pruning in the Spatiotemporal Search stage. The two variants use identical parameter settings, so the comparison isolates the effect of pruning. Here, "w/o pruning" performs Spatiotemporal Search directly on the full AV-specific sub-TopoGraph $\mathcal{G}_s$, whereas IDS uses the proposed prune-TopoGraph. The "S-500" and "S-1000" scenarios denote single-AV routing scenarios with start–destination separations of approximately 500 m and 1000 m, respectively, while M1–Y2 correspond to the random-instruction versions of the port-scale scenarios in Table III. The results show that direct search on the full sub-TopoGraph is computationally infeasible in medium and large scenarios: the no-pruning variant fails to finish within the 1 s threshold from "S-1000" onward and in all port-scale scenarios. By contrast, IDS with prune-TopoGraph completes all tested routing requests within the sub-second response budget. This result supports the design choice in Subsection VII-A: prune-TopoGraph is not intended to preserve every detour alternative in $\mathcal{G}_s$, but to provide a bounded corridor for local spatiotemporal coordination after Direction Search has selected a priority-aware static path. Thus, IDS separates macroscopic detour selection from local time-dependent coordination and deliberately trades full detour completeness for real-time tractability.

TABLE IX
RUNTIME (MILLISECONDS) COMPARISON FOR EVALUATING TOPOGRAPH PRUNING. TO DENOTES TIMEOUT OR FAILURE TO TERMINATE WITHIN 1,000 MS.

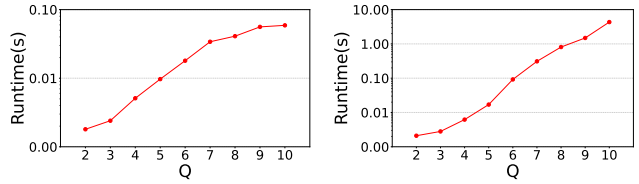
Method	S-500	S-1000	M1	M2	M3	Y1	Y2
w/o pruning	5.36	TO	TO	TO	TO	TO	TO
IDS	6.32	7.87	369.16	373.08	595.65	332.14	424.56

H. Coefficient Analysis

This subsection examines the sensitivity of IDS to two key parameters: Q, the maximum number of allowed left/right lane changes, and H, the maximum rank at which an AV can be approximated as having high routing priority. For each

parameter, we conduct controlled parametric studies across its admissible range to quantify its effect.

Figure 10 reports the runtime for a single Spatiotemporal Search performed by an AV under varying values of Q in the "S-500" and "S-1000" scenarios. To isolate the intrinsic computational cost, the Spatiotemporal Search stage disables periodic clearing of the search space (Algorithm 1, Lines 11–15). As shown in Figure 10, the runtime grows exponentially with Q. In particular, for "S-1000", a single search exceeds 1 second when $Q \geq 9$. This exponential dependence on Q is consistent with, and further corroborates, the IDS time-complexity analysis presented in Subsection VII-D.



(a) Runtime in S-500 (b) Runtime in S-1000

Fig. 10. Runtime comparison of Spatiotemporal Search under different Q.

Table X reports IDS performance across different values of H. As shown, performance initially improves as H increases and then declines once H becomes sufficiently large. Simulation evidence suggests that the magnitude of H influences operational efficiency through two mechanisms:

- When H is small, only a few AVs are treated as having high routing priority. These AVs experience limited interference from other vehicles while traversing the road network; however, variability in crane operation duration increases the likelihood of cranes becoming idle.
- When H is large, more AVs select distance-optimal routes to the cranes. This intensifies queuing in working lanes and increases the incidence of intersection blockages (see Figure 1), which in turn delays AVs and again leads to idle cranes.

Future work will refine parameter tuning by assigning machine-specific values of H to individual working machines. In practical deployments, H will also be dynamically adapted to fluctuations in crane operation durations.

TABLE X
CRANE EFFICIENCY (TEU/HOUR) UNDER DIFFERENT VALUES OF H.

H	Scenario-Instruction			
	M1-Rand	M3-Real	Y1-Rand	Y2-Real
2	50.16	41.22	50.72	42.80
3	**52.38**	43.64	50.80	43.88
4	52.28	**43.80**	**51.66**	**43.94**
5	51.34	43.78	50.88	43.92

IX. DISCUSSION

IDS is designed for closed industrial transportation environments in which a centralized Routing Module can obtain routing requests, HD-map information, and the current operating states of AVs. This setting is consistent with automated ports and mines, where global coordination is typically supported by industrial operating systems and fixed communication infrastructure. Partially decentralized or communication-limited settings are outside the experimental scope of this paper and would require additional mechanisms for regional coordination, information synchronization, and delayed-state handling.

The prune-TopoGraph used in Spatiotemporal Search reflects a practical tradeoff between search completeness and real-time tractability. Searching directly on the full AV-specific sub-TopoGraph would preserve more detour alternatives, but the larger branching factor around intersections would make time-dependent search incompatible with the sub-second response requirement of the Routing Module. IDS therefore performs local spatiotemporal coordination within the priority-aware corridor produced after Direction Search, which is appropriate for real-time industrial routing but may need to be extended in environments where long detours are frequently required for conflict resolution.

The present evaluation estimates energy consumption using an empirical model fitted from real-world operation records and indicates that IDS improves operational efficiency while reducing estimated energy consumption on average. Nevertheless, the current estimate is based on aggregate travel distance and vehicle-hours; finer-grained energy-aware routing would require speed profiles, acceleration histories, waiting states, battery state, and auxiliary power consumption. Finally, although the experiments are conducted in two port environments, the modeling assumptions of IDS are not port-specific. The framework can be adapted to other closed industrial layouts, such as mines, factories, and airports, by redefining working machines, working lanes, operation durations, and scenario-specific traffic rules; validation in these environments is left for future work.

X. CONCLUSION

This paper analyzes the operational characteristics of industrial autonomous driving systems and proposes IDS, a three-stage search method for improving multi-AV routing efficiency in large-scale industrial scenarios. IDS incorporates practical autonomous-driving constraints, including TopoGraph-based routing, request-level block information, routing-priority-based coordination, and real-time response requirements. Experiments in two real-world port environments demonstrate that IDS improves crane efficiency and total operation time compared with existing routing and scheduling methods, supports simultaneous routing for up to 110 AVs within 1 second, and reduces estimated energy consumption on average. Future work will focus on practical deployment, finer-grained energy-aware routing, and validation in broader industrial environments.

REFERENCES

[1] Wang Q, Veerapaneni R, Wu Y, et al., "MAPF in 3D warehouses: dataset and analysis," in *Proceedings of the International Conference on Automated Planning and Scheduling*, 2024, 34: 623-632.

[2] Skrynnik A, Andreychuk A, Yakovlev K, et al., "Decentralized monte carlo tree search for partially observable multi-agent pathfinding," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024, 38(16): 17531-17540.

[3] Zheng H, Ma Y, Araki B, et al., "Learning-guided prioritized planning for lifelong multi-agent path finding in warehouse automation," *Journal of Artificial Intelligence Research*, 2026, 85: 1-30.

[4] Kou N M, Peng C, Ma H, et al., "Idle time optimization for target assignment and path finding in sortation centers," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020, 34(06): 9925-9932.

[5] Tang G, Tang C, Claramunt C, et al., "Geometric A-star algorithm: An improved A-star algorithm for AGV path planning in a port environment," *IEEE Access*, 2021, 9: 59196-59210.

[6] Zhang H, Zhang R, Chen C, et al., "A priority-based autonomous intersection management (AIM) scheme for connected automated vehicles (CAVs)," *Vehicles*, 2021, 3(3): 533-544.

[7] Ahmed H U, Huang Y, Lu P, et al., "Technology developments and impacts of connected and autonomous vehicles: An overview," *Smart Cities*, 2022, 5(1): 382-404.

[8] Rana M M, Hossain K, "Connected and autonomous vehicles and infrastructures: A literature review," *International Journal of Pavement Research and Technology*, 2023, 16(2): 264-284.

[9] Chen L, Englund C, "Cooperative intersection management: A survey," *IEEE Transactions on Intelligent Transportation Systems*, 2015, 17(2): 570-586.

[10] Namazi E, Li J, Lu C, "Intelligent intersection management systems considering autonomous vehicles: A systematic literature review," *IEEE Access*, 2019, 7: 91946-91965.

[11] Stern R, Sturtevant N, Felner A, et al., "Multi-agent pathfinding: Definitions, variants, and benchmarks," in *Proceedings of the International Symposium on Combinatorial Search*, 2019, 10(1): 151-158.

[12] Varambally S, Li J, Koenig S, "Which MAPF model works best for automated warehousing?" in *Proceedings of the International Symposium on Combinatorial Search*, 2022, 15(1): 190-198.

[13] Andreychuk A, Yakovlev K, Panov A, et al., "Mapf-gpt: Imitation learning for multi-agent pathfinding at scale," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, 39(22): 23126-23134.

[14] Li B, Chen K, Song Z, et al., "Divide and Conquer: Advancing Large-Scale Multi-Agent Pathfinding with Hierarchical Reinforcement Learning," *IEEE Transactions on Mobile Computing*, 2025.

[15] Wang Y, Duhan T, Li J, et al., "LNS2+ RL: Combining multi-agent reinforcement learning with large neighborhood search in multi-agent path finding," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, 39(22): 23343-23350.

[16] Okumura K, "Lacam: Search-based algorithm for quick multi-agent pathfinding," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023, 37(10): 11655-11662.

[17] Song Q, Engström E, Runeson P, "Industry practices for challenging autonomous driving systems with critical scenarios," *ACM Transactions on Software Engineering and Methodology*, 2024, 33(4): 1-35.

[18] Omeiza D, Webb H, Jirotka M, et al., "Explanations in autonomous driving: A survey," *IEEE Transactions on Intelligent Transportation Systems*, 2021, 23(8): 10142-10162.

[19] Shouwen J, Di L, Zhengrong C, et al., "Integrated scheduling in automated container terminals considering AGV conflict-free routing," *Transportation Letters*, 2021, 13(7): 501-513.

[20] Hu H, Yang X, Xiao S, et al., "Anti-conflict AGV path planning in automated container terminals based on multi-agent reinforcement learning," *International Journal of Production Research*, 2023, 61(1): 65-80.

[21] Kato S, Tokunaga S, Maruyama Y, et al., "Autoware on board: Enabling autonomous vehicles with embedded systems," in *Proceedings of the International Conference on Cyber-Physical Systems*, 2018: 287-296.

[22] Fan H, Zhu F, Liu C, et al., "Baidu apollo em motion planner," arXiv preprint arXiv:1807.08048, 2018.

[23] Li J, Lin E, Vu H L, et al., "Intersection coordination with priority-based search for autonomous vehicles," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023, 37(10): 11578-11585.

[24] Li P T, Zhou X, "Recasting and optimizing intersection automation as a connected-and-automated-vehicle (CAV) scheduling problem: A sequential branch-and-bound search approach in phase-time-traffic hypernetwork," *Transportation Research Part B: Methodological*, 2017, 105: 479-506.

[25] Zhao R, Li Y, Wang K, et al., "Centralized cooperation for connected autonomous vehicles at intersections by safe deep reinforcement learning," *IEEE Transactions on Mobile Computing*, 2024, 23(12): 12830-12847.

[26] Wang J, Guo X, Yang X, "Efficient and safe strategies for intersection management: a review," *Sensors*, 2021, 21(9): 3096.

[27] Ma H, Harabor D, Stuckey P J, et al., "Searching with consistent prioritization for multi-agent path finding," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019, 33(01): 7643-7650.

[28] Xu H, Zhang Y, Li L, et al., "Cooperative driving at unsignalized intersections using tree search," *IEEE Transactions on Intelligent Transportation Systems*, 2019, 21(11): 4563-4571.

[29] Li H, Dong W, Lu L, et al., "Distributed Cooperative Driving Strategy for Connected Automated Vehicles at Unsignalized Intersections Based on Monte Carlo Method," *Journal of Advanced Transportation*, 2024, 2024(1): 6586774.

[30] El Zorkany M, Yasser A, Galal A I, "Vehicle to vehicle 'V2V' communication: scope, importance, challenges, research directions and future," *The Open Transportation Journal*, 2020, 14(1).

[31] Pan W, Lin B, Wang Y, et al., "Cooperative Driving at Multiple Unsignalized Intersections in Fully Autonomous Driving Scenarios," *IEEE Transactions on Intelligent Transportation Systems*, 2025.

[32] Lakas A, Ferrag M A, Debbah M., "LISA: Cognitive Arbitration for Signal-Free Autonomous Intersection Management," arXiv preprint arXiv:2605.12321, 2026.

[33] Erdmann M, Lozano-Perez T, "On multiple moving objects," *Algorithmica*, 1987, 2(1): 477-521.

[34] Dresner K, Stone P, "A multiagent approach to autonomous intersection management," *Journal of Artificial Intelligence Research*, 2008, 31: 591-656.

[35] Phillips M, Likhachev M, "Sipp: Safe interval path planning for dynamic environments," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2011: 5628-5635.

[36] Levin M W, Rey D, "Conflict-point formulation of intersection control for autonomous vehicles," *Transportation Research Part C: Emerging Technologies*, 2017, 85: 528-547.

[37] Yao Z, Zhao Y, Jiang H, et al., "A Review of Cooperative Intersection: From Design to Management," *IEEE Transactions on Intelligent Vehicles*, 2025.

[38] Ebrahimi Soorchaei B, Razzaghpour M, Valiente R, et al., "High-definition map representation techniques for automated vehicles," *Electronics*, 2022, 11(20): 3374.

[39] Skrynnik A, Andreychuk A, Nesterova M, et al., "Learn to follow: Decentralized lifelong multi-agent pathfinding via planning and learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024, 38(16): 17541-17549.

[40] Chen Z, Harabor D, Li J, et al., "Traffic flow optimisation for lifelong multi-agent path finding," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024, 38(18): 20674-20682.

[41] Zhang X, Li J, Zhou J, et al., "Vehicle-to-Everything Communication in Intelligent Connected Vehicles: A Survey and Taxonomy," *Automotive Innovation*, 2025: 1-33.

[42] Chalaki B, Malikopoulos A A, "Time-optimal coordination for connected and automated vehicles at adjacent intersections," *IEEE Transactions on Intelligent Transportation Systems*, 2021, 23(8): 13330-13345.

[43] On-Road Automated Driving (ORAD) Committee, "Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles," *SAE International*, 2021.

[44] Sell R, Malayjerdi M, Pikner H, et al., "Open-source level 4 autonomous shuttle for last-mile mobility," in *Proceedings of the IEEE 29th International Conference on Emerging Technologies and Factory Automation*, 2024: 01-06.

[45] Veerapaneni R, Saleem M S, Li J, et al., "Windowed MAPF with Completeness Guarantees," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, 39(22): 23323-23332.

[46] Ali Z A, Yakovlev K, "Safe interval path planning with kinodynamic constraints," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023, 37(10): 12330-12337.

[47] Standley T, "Finding optimal solutions to cooperative pathfinding problems," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2010, 24(1): 173-178.

[48] Grafarend E W, "The optimal universal transverse Mercator projection," *Manuscripta geodaetica*, 1995, 20(6): 421-468.

[49] De Cauwer C, Verbeke W, Coosemans T, et al., "A data-driven method for energy consumption prediction and energy-efficient routing of electric vehicles in real-world conditions," *Energies*, 2017, 10(5): 608.



Zekuan Fang is currently pursuing the Ph.D. degree with the School of Software Technology, Zhejiang University, Ningbo 315100, China. Before that, he received the B.S. degree in automation from Zhejiang University of Technology, China, in 2020, and the M.S. degree in software engineering from East China Normal University, China, in 2023. His research interests include autonomous driving, reinforcement learning, and multimodal learning.



Kaihao Lan received the M.S. degree in computer technology from Nanjing University of Science and Technology in 2023. He is currently a Research and Development Engineer at FABU Technology Co., Ltd., where he focuses on the research and development of autonomous driving systems. His research interests include 3D computer vision and autonomous driving technologies.



Mengpeng He received the B.S. and M.S. degrees in computer science and technology from Zhejiang University in 2013 and 2016, respectively. During his graduate studies, his research focused on artificial intelligence, deep learning, and computer vision. From 2016 to 2017, he worked as a Researcher at Didi Technology, focusing on autonomous driving and planning algorithms. In 2018, he joined FABU Technology Co., Ltd., where he has been engaged in the research and development of autonomous driving systems. His current research interests include multi-vehicle coordination, intelligent scheduling, path planning, and autonomous driving systems.



Tianhao Jiang received the B.S. degree from the School of Computer Science at Hangzhou Dianzi University in 2019. Since 2022, he has worked as a C++ Engineer at FABU Technology Co., Ltd., focusing on the development and optimization of motion planning modules for autonomous driving. His research interests include autonomous driving, motion planning, trajectory optimization, and multi-agent decision-making.



Zheng Yang received the B.S. degree in network engineering from Sun Yat-sen University, Guangzhou, China, in 2010, and the Ph.D. degree in computer science and technology from Zhejiang University, Hangzhou, China, in 2016. He was an Algorithm Engineer at DiDi Chuxing, where he worked on ride-hailing optimization and autonomous driving. In 2017, he co-founded FABU Technology Co., Ltd., where he has served as the Chief Technology Officer. His research interests include vehicle scheduling and fleet optimization, 3D perception, end-to-end driving, and foundation models for autonomous driving.



Liang Xie received the Ph.D. degree in computer science and technology from Zhejiang University in 2023. He is currently a Lecturer with Zhejiang University of Technology, China. His research interests include large language models, multimodal artificial intelligence, explainable AI, trustworthy machine learning, and 3D computer vision.



Weihang Pan is currently a Postdoctoral Researcher at the School of Software Technology, Zhejiang University, Ningbo 315100, China, with research interests in agentic reinforcement learning, autonomous agents, and multi-agent systems.



Ziqi Wang (Student Member, IEEE) is currently pursuing the Ph.D. degree with the School of Software Technology, Zhejiang University, Ningbo 315100, China. Before that, he received the B.S. degree in internet of things in 2022 and the M.S. degree in software engineering in 2025, both from Beijing University of Technology, China. His research interests include mobile edge computing, task scheduling, intelligent optimization algorithms, and deep learning. He received the Best Paper Award at the 2024 ICAIS & ISAS and the Best Application Paper Award at the 21st IEEE ICNSC.



Binbin Lin is an Associate Professor with the School of Software Technology, Zhejiang University, Ningbo 315100, China. He received the Ph.D. degree in computer science from Zhejiang University in 2012. His research interests include machine learning and decision making.